

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
CURSO DE ENGENHARIA DE COMPUTAÇÃO

VICTOR DE SOUZA VILELA DA SILVA

INTERPRETADOR DINÂMICO PARA ENSINO DE PROGRAMAÇÃO:
Implementação de análise léxica e sintática, geração e interpretação de código
intermediário em ambiente web

Leopoldina

2026

A326m Silva, Victor de Souza Vilela da.

2026 Interpretador Dinâmico para Ensino de Programação : Implementação de análise léxica e sintática, geração e interpretação de código intermediário em ambiente web / Victor de Souza Vilela da Silva

Vilela - Leopoldina : CEFET-MG, .

viii, 63 f. : il.

Orientador: Luís Augusto Mattos Mendes

Coorientador: Eduardo Gabriel Reis Miranda

TCC (Graduação) – Centro Federal de Educação Tecnológica de Minas Gerais, Unidade Leopoldina. Engenharia de Computação.

1. Interpretador dinâmico. 2. Ensino de programação. 3. Compiladores. 4. Análise léxica. 5. Código intermediário. I. Mendes, Luís Augusto Mattos. II. Miranda, Eduardo Gabriel Reis. III. Interpretador dinâmico para ensino de programação: implementação de análise léxica e sintática, geração e interpretação de código intermediário em ambiente web.

CDU 004.414.23

VICTOR DE SOUZA VILELA DA SILVA

**INTERPRETADOR DINÂMICO PARA ENSINO DE PROGRAMAÇÃO :
Implementação de análise léxica e sintática, geração e interpretação de código
intermediário em ambiente web**

Trabalho de conclusão de curso apresentado ao
Curso de Engenharia de Computação do Centro
Federal de Educação Tecnológica de Minas
Gerais, do Campus Leopoldina, como parte do
requisito para obtenção do título de Bacharel em
Engenharia de Computação.

Aprovado em: de de

Banca Examinadora:

Prof. Msc. Luís Augusto Mattos Mendes- Orientador
CEFET-MG

Prof. Dsc. Eduardo Gabriel Reis Miranda- Coorientador
CEFET-MG

Prof. Msc. Matheus Ávila Moreira de Paula
CEFET-MG

Prof. Dsc. Gabriella Castro Barbosa Costa
CEFET-MG

(Assinaturas Digitais)

**Leopoldina
2026**

AGRADECIMENTOS

Agradeço à minha família pelo apoio incondicional, incentivo e paciência ao longo desta jornada. Sem vocês, nada disso seria possível. Meu sincero agradecimento também ao meu orientador, Luís Augusto Mattos Mendes, e ao meu coorientador, Eduardo Gabriel Reis Miranda, pela orientação, dedicação e ensinamentos valiosos que tornaram este trabalho possível. A todos que, de alguma forma, contribuíram para esta conquista, meu muito obrigado.

RESUMO

O ensino de programação apresenta diversos desafios, em parte devido ao uso de linguagens cuja escrita é definida por sintaxe fixa e palavras-chave reservadas. Isso pode dificultar a assimilação por parte dos estudantes que, mesmo antes de compreender a estrutura dos algoritmos, precisam memorizar termos em outro idioma e conjuntos de regras rígidas. Este trabalho propõe uma plataforma educacional com um Ambiente de Desenvolvimento Integrado (IDE) conectado a um interpretador dinâmico que permite ao usuário personalizar a sintaxe da linguagem, incluindo as palavras reservadas, os operadores e as regras estruturais. O projeto é desenvolvido por meio de revisão de literatura, projeto da arquitetura do sistema e implementação de uma IDE web e de um interpretador, que inclui análise léxica e sintática, geração e interpretação de código intermediário, com execução em tempo real no navegador. Espera-se como resultado que a plataforma atue como facilitadora do processo de ensino-aprendizagem, reduzindo as barreiras linguísticas iniciais e incentivando a experimentação, de modo que os alunos possam concentrar-se no raciocínio algorítmico em vez da memorização de estruturas rígidas de linguagem. O diferencial da proposta está em permitir a redefinição flexível do vocabulário e das regras estruturais da linguagem, adaptando-a ao repertório do estudante.

Palavras-chave: Interpretador dinâmico; Educação em Programação Introdutória; Programação Orientada à Linguagem; Ambiente de Programação Interativo; ensino-aprendizagem.

ABSTRACT

Teaching programming presents several challenges, partly due to the use of languages whose writing is defined by fixed syntax and reserved keywords. This can make assimilation difficult for students who, even before understanding the structure of algorithms, must memorize terms in another language and sets of rigid rules. This work proposes an educational platform with an Integrated Development Environment (IDE) connected to a dynamic interpreter that allows the user to customize the language syntax, including its reserved words, operators, and structural rules. The project is developed through literature review, system architecture design, and the implementation of a web IDE and an interpreter, which includes lexical and syntactic analysis, intermediate code generation and interpretation, with real-time execution in the browser. The expected result is that the platform acts as a facilitator of the teaching-learning process, reducing initial linguistic barriers and encouraging experimentation, so that students can focus on algorithmic reasoning rather than memorizing rigid language structures. The main contribution of the proposal lies in allowing the flexible redefinition of the language's vocabulary and structural rules, adapting it to the student's background.

Keywords: Dynamic Interpreter. Introductory Programming Education. Language-Oriented Programming. Interactive Programming Environment. Teaching-Learning.

LISTA DE ILUSTRAÇÕES

Figura 2.1 –Arquitetura de um compilador moderno.	14
Figura 2.2 –Exemplificação de agrupamento de caracteres em lexemas.	16
Figura 2.3 –Ciclo de Desenvolvimento Guiado por Testes (TDD).	27
Figura 4.1 –Fluxo de execução entre os módulos do interpretador dinâmico.	42
Figura 4.2 –Esquema lógico do banco de dados da plataforma.	44
Figura 4.3 –Funções de validação prévia da configuração léxica da linguagem.	49

LISTA DE TABELAS

Tabela 3.1 – Cronograma de desenvolvimento do trabalho de conclusão de curso.	38
Tabela 4.1 – Analisadores especializados (<i>scanners</i>) despachados pela <code>LexerScannerFactory</code> . 51	
Tabela B.1 – Grupos de recursos da API da plataforma.	62
Tabela C.1 – Distribuição dos casos de teste por camada do compilador.	63

SUMÁRIO

1	INTRODUÇÃO	10
1.1	Justificativa	11
1.2	Objetivos	11
1.2.1	Objetivo Geral	11
1.2.2	Objetivos Específicos	11
1.3	Estrutura do Trabalho	12
2	REFERENCIAL TEÓRICO E TRABALHOS RELACIONADOS . . .	13
2.1	Compiladores	13
2.1.1	Compiladores e seus Fundamentos	13
2.1.2	O Modelo de Análise e Síntese (Fases da Compilação)	14
2.1.3	A Análise Léxica e o Reconhecimento de <i>Tokens</i>	16
2.1.4	Estratégias de Análise Sintática: <i>Top-Down</i> e <i>Bottom-Up</i>	18
2.1.5	Análise Semântica e Verificação de Tipos	19
2.1.6	Geração de Código Intermediário	20
2.1.7	Otimização de Código e Eficiência Computacional	22
2.1.8	Geração de Código Objeto e Mapeamento de Arquitetura	23
2.1.9	Engenharia de Software	24
2.1.9.1	Gestão de Tarefas e Metodologias Ágeis	25
2.1.9.2	Testes Automatizados e <i>Test-Driven Development</i> (TDD)	26
2.1.9.3	Arquitetura Limpa e Separação de Responsabilidades	28
2.1.9.4	Qualidade de Código e <i>Clean Code</i>	29
2.1.9.5	FastAPI: Framework para APIs de Alto Desempenho	30
2.2	Softwares Educativos e Tecnologias de Apoio à Aprendizagem	31
2.3	Trabalhos Relacionados	32
2.3.1	Portugol Studio e Portugol Webstudio	32
2.3.2	Laila: Gerador de Analisadores Léxicos e Sintáticos Online	33
2.3.3	Quorum: Linguagem de Programação Baseada em Evidências	34
2.3.4	chibicc: Uma Abordagem Incremental para o Ensino de Compiladores	35
2.3.5	Beecrowd: Prática de Algoritmos e Avaliação Automatizada	35
2.3.6	Análise Crítica e o Diferencial da Solução Proposta	36

3	PROPOSTA	38
4	METODOLOGIA	40
4.1	Seleção dos Trabalhos Relacionados	40
4.2	Modelagem da Arquitetura do Projeto	41
4.2.1	Modelagem do Banco de Dados	43
4.2.2	Especificação das Rotas da API	46
4.3	Especificação da Linguagem e de seus Elementos Configuráveis	47
4.4	Interpretador	49
4.4.1	Análise Léxica	50
4.4.2	Análise Sintática	51
4.4.3	Análise Semântica	52
4.4.4	Geração de Código Intermediário	53
4.4.5	Interpretação e Tratamento de Erros em Tempo de Execução	54
4.5	Integração entre a Plataforma e o Interpretador	55
4.6	Estratégia de Testes Automatizados	56
	REFERÊNCIAS	57
A	PLANO DE TRABALHO: DIVISÃO DE RESPONSABILIDADES . . .	60
A.1	Atividades Desenvolvidas em Regime Colaborativo	60
A.2	Eixos de Aprofundamento Individual	60
A.2.1	Victor de Souza Vilela da Silva (esta monografia)	60
A.2.2	Igor Lamoia Queiroz (documento complementar)	61
B	DOCUMENTAÇÃO DA API	62
C	CASOS DE TESTE AUTOMATIZADOS	63

1 INTRODUÇÃO

Este Trabalho de Conclusão de Curso (TCC) foi desenvolvido em parceria com Igor Lamoia Queiroz, aluno do Curso de Engenharia de Computação do CEFET-MG, Unidade Leopoldina. Os dois trabalhos tratam do mesmo projeto, mas com enfoques distintos. Este volume descreve o núcleo do compilador: as fases de análise léxica, análise sintática, geração de código intermediário e execução pelo interpretador. O trabalho complementar, intitulado *Interpretador Dinâmico para Ensino de Programação: Uma abordagem interativa com Ambiente de Desenvolvimento Integrado (IDE) para personalização e execução de comandos definidos pelo usuário*, foca na IDE web, na interface de personalização da linguagem e na experiência de uso da plataforma. Por se tratar de trabalhos complementares e interdependentes, para a compreensão do sistema como um todo, recomenda-se ler os dois trabalhos em conjunto.

A contextualização do problema de pesquisa e as premissas sobre os desafios no ensino e na aprendizagem de programação, que fundamentam esta introdução e suas subseções, baseiam-se nas investigações de Stefik e Siebert (2013) e Ferreira, Silva e Ribeiro (2025).

Programar virou uma habilidade cada vez mais necessária, não só em cursos de tecnologia, mas em várias outras áreas. Mesmo assim, disciplinas de programação ainda registram altos índices de reprovação e evasão, principalmente nos primeiros semestres (SOUZA; BATISTA; BARBOSA, 2016; SILVA et al., 2020). Um dos motivos mais frequentes é que os alunos precisam aprender a lógica de programação e a sintaxe de uma linguagem ao mesmo tempo, o que aumenta a dificuldade do processo de aprendizagem (STEFIK; SIEBERT, 2013).

Em linguagens comerciais, um simples erro de digitação, a falta de um ponto e vírgula ou o uso incorreto de uma palavra reservada já são suficientes para interromper a compilação e gerar mensagens de erro difíceis de entender para quem está começando. O aluno, que deveria estar pensando no algoritmo, acaba gastando tempo e energia tentando descobrir o que está sintaticamente errado (STEFIK; SIEBERT, 2013).

Diante disso, este trabalho parte da seguinte pergunta: é possível reduzir a barreira de aprendizagem a partir da personalização da sintaxe da linguagem que o aluno está aprendendo?

A hipótese é que sim. Se o aluno puder definir as palavras-chave, os delimitadores de bloco e os operadores de uma forma que faça sentido para ele, a curva de aprendizagem tende a ser mais suave. O foco passa a ser o raciocínio algorítmico, não a memorização de regras gramaticais de uma linguagem estrangeira.

1.1 Justificativa

Já existem plataformas e linguagens voltadas para iniciantes, como o Portugol Studio e o Scratch. A grande maioria, porém, trabalha com uma gramática fixa definida pelos desenvolvedores da ferramenta. O aluno não tem como ajustar ou adaptar a linguagem ao próprio raciocínio.

A proposta deste trabalho é oportunizar a mudança desse cenário a partir da construção de um interpretador que aceite regras gramaticais configuradas pelo próprio usuário, integrado a uma IDE web que permita uma personalização simples e acessível. Além de diminuir o obstáculo inicial com a sintaxe, a plataforma funciona como um ambiente de experimentação antes de o aluno migrar para linguagens mais rígidas exigidas pelo mercado de trabalho e por plataformas de juízes online.

1.2 Objetivos

A seguir são apresentados o objetivo geral e os objetivos específicos que nortearam o desenvolvimento desta pesquisa e a construção da ferramenta.

1.2.1 Objetivo Geral

Projetar e desenvolver um interpretador dinâmico integrado a uma IDE web que permita ao usuário personalizar a sintaxe e o vocabulário da linguagem, com o objetivo de facilitar o processo de ensino-aprendizagem de algoritmos para estudantes iniciantes.

1.2.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Estudar os fundamentos de construção de compiladores;
- Analisar ferramentas educacionais correlatas na literatura;
- Projetar os módulos de análise léxica, sintática e semântica com suporte à injeção dinâmica de regras gramaticais;
- Implementar os módulos projetados, integrando-os em um interpretador dinâmico executável no navegador;

- Integrar o núcleo do interpretador a uma interface web, definindo os contratos de comunicação entre os módulos;
- Avaliar tecnicamente a solução desenvolvida por meio de estudos de caso centrados na validade do núcleo de tradução.

1.3 Estrutura do Trabalho

Este documento está organizado da seguinte forma: o Capítulo 2 apresenta o referencial teórico, cobrindo os conceitos de compiladores, metodologias de software e trabalhos relacionados. O Capítulo 3 descreve a proposta do trabalho e o cronograma nas duas etapas do TCC. Por fim, o Capítulo 4 detalha a modelagem da arquitetura e a implementação realizada até o encerramento do TCC I, incluindo o núcleo do interpretador, a integração com a plataforma web e a estratégia de testes.

2 REFERENCIAL TEÓRICO E TRABALHOS RELACIONADOS

Este capítulo reúne a fundamentação teórica que sustenta o trabalho. Inicialmente, a Seção 2.1 apresenta os fundamentos de compiladores e suas fases, da análise léxica à geração de código intermediário, bem como os conceitos de engenharia de software que orientaram a construção do sistema. Em seguida, discutem-se os softwares educativos e as tecnologias de apoio à aprendizagem de programação. Por fim, a Seção 2.3 analisa os trabalhos relacionados, posicionando a solução proposta frente ao estado da arte.

2.1 Compiladores

Esta seção percorre os fundamentos de compiladores que embasam o núcleo de tradução implementado neste trabalho, desde a definição de compilador e de suas fases até a geração de código intermediário.

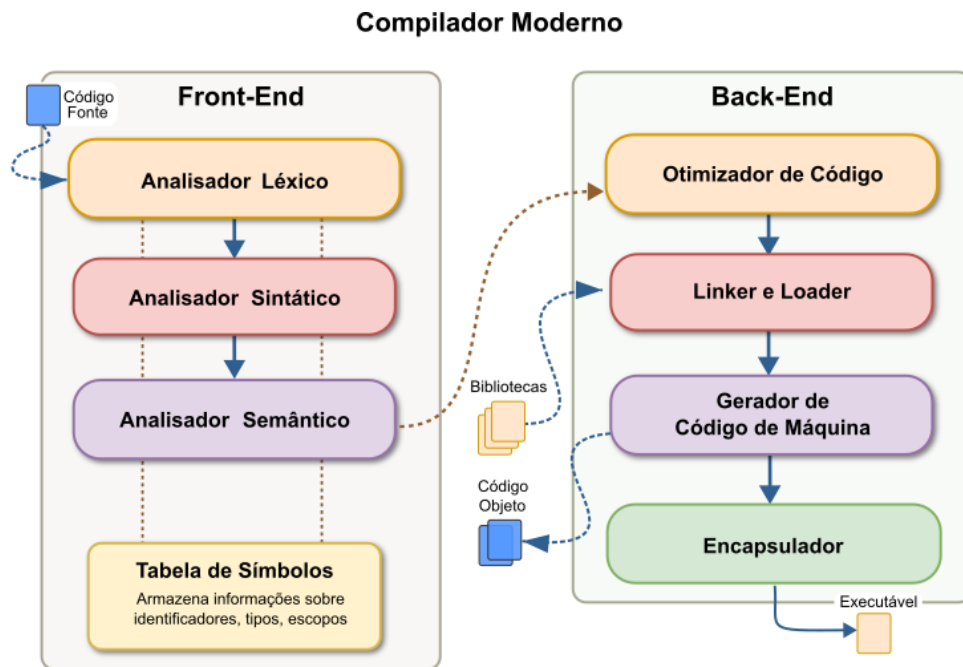
2.1.1 Compiladores e seus Fundamentos

Para a fundamentação teórica acerca do funcionamento, das fases e da arquitetura dos tradutores de linguagem, esta subseção adota como base estrutural a obra clássica de Aho, Sethi e Ullman (1995), amplamente reconhecida na literatura de ciência da computação como a principal referência para o projeto de compiladores.

Posto de forma simples, um compilador é um programa de computador que lê um programa escrito em uma linguagem fonte e o traduz para um programa equivalente em uma linguagem alvo. O modelo de compilação moderno é estruturado dividindo esse processo em duas partes fundamentais: a análise e a síntese. A fase de análise, frequentemente associada ao *Front-End*, divide o programa fonte em suas partes constituintes e cria uma representação intermediária do mesmo. Por outro lado, a fase de síntese, ou *Back-End*, constrói o programa alvo desejado a partir dessa representação intermediária gerada (AHO; SETHI; ULLMAN, 1995).

A Figura 2.1 ilustra essa separação lógica e a progressão da transformação do código.

Figura 2.1 – Arquitetura de um compilador moderno.



Fonte: Alcantara (2024).

Conceitualmente, um compilador opera em fases sequenciais, cada uma das quais transforma o programa fonte de uma representação abstrata para outra. As primeiras fases formam o núcleo de análise e englobam a análise léxica, a análise sintática e a análise semântica. A partir daí, o fluxo avança para as etapas de geração de código intermediário, otimização e geração de código alvo.

Interagindo com todas essas seis fases de compilação, encontra-se o gerenciamento da tabela de símbolos. Essa estrutura de dados é essencial para o compilador, pois atua registrando todos os identificadores usados no programa e coletando informações cruciais sobre seus atributos, permitindo rápida recuperação de escopos e validação semântica ao longo de todo o processo de tradução (AHO; SETHI; ULLMAN, 1995).

2.1.2 O Modelo de Análise e Síntese (Fases da Compilação)

Toda a estruturação das fases da compilação e os conceitos detalhados nesta seção fundamentam-se nas obras clássicas da área, notadamente nos estudos de Aho, Sethi e Ullman (1995), Price e Toscani (2001) e Cooper e Torczon (2012).

O processo de compilação é tradicionalmente dividido em duas partes principais: a

análise e a síntese. A fase de análise, frequentemente chamada de *front-end*, concentra-se em compreender o programa fonte dividindo o código em seus elementos constituintes para criar uma representação intermediária abstrata. Em contrapartida, a fase de síntese, ou *back-end*, foca em mapear e construir o programa alvo desejado a partir dessa representação intermediária. A separação entre *front-end* e *back-end* atua como uma interface que facilita a adaptação e o suporte a múltiplas linguagens e arquiteturas (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

Para dominar a complexidade da tradução de uma linguagem de alto nível, o fluxo geral é subdividido em módulos lógicos menores, conhecidos como fases, onde cada uma transforma a representação do programa para a etapa seguinte. Em uma implementação prática, as atividades lógicas dessas fases podem ser agrupadas em "passagens" (*passes*), otimizando o uso de memória durante o processo. As principais fases que compõem o núcleo de um compilador são:

- **Análise Léxica (*Scanner*):** Constitui a primeira fase da compilação, fazendo a leitura do programa fonte caractere por caractere. Sua função é traduzir o texto para uma sequência de agrupamentos lógicos chamados de *tokens*, ao mesmo tempo em que descarta caracteres não significativos, como espaços em branco e comentários.
- **Análise Sintática (*Parser*):** Recebe a cadeia de *tokens* do analisador léxico e verifica se a estrutura gramatical do programa está correta. O *parser* agrupa os *tokens* hierarquicamente, produzindo uma estrutura de dados conhecida como árvore de derivação ou árvore de sintaxe, seja através de estratégias *top-down* (descendentes) ou *bottom-up* (redutivas).
- **Análise Semântica:** Vai além da verificação estrutural sintática, realizando checagens para garantir que os componentes do programa fazem sentido e combinam-se de forma válida durante a execução. Um componente crítico desta fase é a verificação de tipos (*type checking*), que assegura a consistência lógica do uso de variáveis e operadores.
- **Geração de Código Intermediário:** Após as análises, o compilador gera uma representação intermediária explícita que abstrai os detalhes específicos da máquina alvo, sendo o "código de três endereços" um de seus formatos clássicos. Essa etapa simplifica a implementação geral quebrando a barreira de complexidade entre a linguagem de alto nível e as instruções de baixo nível.

- **Otimização de Código:** Atua sobre o código intermediário com o objetivo de reescrevê-lo e melhorá-lo. Essa fase foca em reduzir o tempo global de execução e minimizar o espaço de memória consumido pelo programa compilado.
- **Geração de Código Objeto:** É a fase final de síntese, responsável por mapear o código intermediário para a linguagem alvo (código de máquina absoluto, relocável ou *assembly*). Ela exige uma seleção cuidadosa das instruções nativas da máquina, das posições de memória e a alocação eficiente dos registradores.

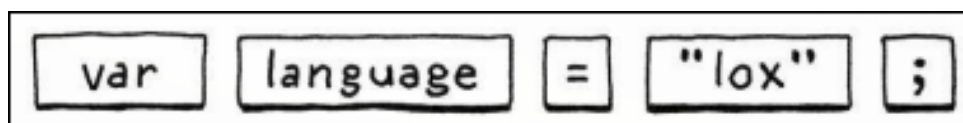
Ao longo de todas as fases, o compilador interage ininterruptamente com dois módulos de suporte vitais: o gerenciador da **Tabela de Símbolos** e o **Atendimento a Erros**. A tabela de símbolos registra os identificadores encontrados e armazena atributos essenciais (como escopo, tipo e métodos de transmissão de parâmetros) de maneira que permita uma recuperação extremamente rápida nas etapas posteriores. Já o módulo de erros atua detectando falhas léxicas, sintáticas ou lógicas em todas as fases, utilizando mecanismos de recuperação que re-sincronizam o analisador e permitem que a compilação prossiga na tentativa de descobrir falhas adicionais no código fonte (AHO; SETHI; ULLMAN, 1995).

2.1.3 A Análise Léxica e o Reconhecimento de *Tokens*

Os conceitos e definições inerentes à fase de análise léxica, detalhados a seguir, fundamentam-se fortemente nas obras de Cooper e Torczon (2012), Nystrom (2021) e Price e Toscani (2001).

A análise léxica, frequentemente denominada *scanner*, constitui a primeira fase do processo de compilação. A função primária deste módulo é realizar a leitura do programa fonte, caractere por caractere, e agrupá-los logicamente em unidades com significado gramatical, denominadas lexemas, conforme exemplificado na Figura 2.2.

Figura 2.2 – Exemplificação de agrupamento de caracteres em lexemas.



Fonte: Nystrom (2021, p. 43).

Durante essa varredura, o analisador léxico também atua filtrando informações que não possuem relevância para as fases posteriores, descartando ativamente espaços em branco,

tabulações e comentários inseridos pelo programador.

Neste contexto, é imprescindível estabelecer a distinção teórica entre dois conceitos fundamentais: o *lexema* e o *token*. Um *lexema* é a sequência exata de caracteres no código-fonte que representa uma unidade léxica, ou seja, é a *string* literal que foi digitada pelo usuário, como por exemplo a sequência “123” ou “var”. Por outro lado, o *token* é a representação lógica ou estrutura de dados gerada a partir desse *lexema*. Ele encapsula a classificação da unidade (sua classe ou categoria sintática, como identificador, palavra reservada ou operador) e, frequentemente, o valor original do *lexema* e sua posição no texto (linha e coluna) para auxiliar na emissão de erros sintáticos e semânticos (NYSTROM, 2021; COOPER; TORCZON, 2012).

Para que o analisador léxico consiga identificar e classificar esses *lexemas* corretamente, a micro-sintaxe da linguagem de programação precisa ser formalmente descrita. Isso é realizado através de expressões regulares, que fornecem uma notação matemática concisa e poderosa para especificar a estrutura exata de cada tipo de *token* (como, por exemplo, a regra que define que um identificador deve começar com uma letra seguida de letras ou dígitos). As expressões regulares descrevem conjuntos de cadeias de caracteres chamados de linguagens regulares, que figuram entre as linguagens mais simples na hierarquia das linguagens formais (PRICE; TOSCANI, 2001).

Embora as expressões regulares sejam excelentes para a especificação humana, o reconhecimento mecânico em tempo de execução exige um formalismo executável. Para a implementação computacional, as expressões regulares são traduzidas para autômatos finitos. Um autômato finito é uma máquina abstrata de transição de estados que atua como um reconhecedor. Ele processa a fita de entrada caractere por caractere, mudando de estado lógico; se, ao final da leitura de um *lexema*, o autômato atingir um estado de aceitação (estado final), o *token* é validado e encaminhado para o analisador sintático.

O uso de autômatos finitos permite que a análise léxica seja extremamente eficiente, com custo de tempo linear em relação ao tamanho do código-fonte. Na prática, geradores de analisadores léxicos, como a clássica ferramenta *Lex*, recebem as expressões regulares definidas pelo projetista da linguagem e geram automaticamente o código fonte de um autômato finito determinístico capaz de reconhecer os *tokens* da linguagem de forma otimizada (COOPER; TORCZON, 2012).

2.1.4 Estratégias de Análise Sintática: *Top-Down* e *Bottom-Up*

As definições e conceitos apresentados nesta seção baseiam-se nos fundamentos estabelecidos por Aho, Sethi e Ullman (1995), Cooper e Torczon (2012), Price e Toscani (2001) e Nystrom (2021).

A fase de análise sintática (ou *parsing*) tem a responsabilidade central de determinar se a cadeia de *tokens* fornecida pelo analisador léxico pode ser validamente gerada pela gramática formal da linguagem fonte. Para realizar essa verificação e construir a árvore de derivação, os compiladores adotam abordagens matemáticas que se enquadram majoritariamente em duas categorias fundamentais de algoritmos: a análise descendente (*top-down*) e a análise ascendente ou redutiva (*bottom-up*) (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Análise Sintática Descendente (*Top-Down*)

A estratégia descendente consiste na tentativa de construir a árvore de derivação a partir da sua raiz (o símbolo inicial da gramática) em direção às folhas (os *tokens* terminais da cadeia de entrada). Em cada passo do processo, o analisador substitui o símbolo não-terminal mais à esquerda por um de seus lados direitos correspondentes nas regras de produção, realizando o que se denomina rigorosamente como uma derivação mais à esquerda (*leftmost derivation*).

Os analisadores *top-down* dividem-se tradicionalmente em preditivos tabulares (como o algoritmo LL(1)) e recursivos descendentes. A técnica de descida recursiva é amplamente utilizada em compiladores de produção modernos em virtude de sua elegância, facilidade de codificação manual e excelente capacidade de detectar e relatar erros de sintaxe. Contudo, para que um analisador descendente funcione de maneira eficiente e evite o custoso retrocesso (*backtracking*), a gramática subjacente da linguagem deve atender a restrições estritas. Especificamente, a gramática não pode conter recursividade à esquerda e deve estar fatorada à esquerda, de modo que o próximo *token* da entrada seja suficiente para guiar univocamente o analisador na escolha da produção correta (NYSTROM, 2021; PRICE; TOSCANI, 2001).

Análise Sintática Ascendente ou Redutiva (*Bottom-Up*)

Em contrapartida à técnica anterior, a análise ascendente processa a estrutura do programa de baixo para cima, iniciando a construção da árvore a partir das folhas (os *tokens* fornecidos pelo *scanner*) e trabalhando estruturalmente em direção à raiz.

A denominação “reduativa” deve-se à natureza do processo: o analisador busca identificar uma sequência de símbolos na pilha que corresponda exatamente ao lado direito de uma regra de produção (o *handle*) e a substitui (reduz) pelo símbolo não-terminal correspondente ao lado esquerdo daquela regra. Esse mecanismo iterativo constrói sistematicamente uma derivação mais à direita ao reverso.

A família mais proeminente e poderosa de analisadores *bottom-up* é a dos analisadores da classe LR (como SLR, LALR e o Canônico), que operam através de uma arquitetura do tipo empilha-reduz (*shift-reduce*) orientada por tabelas de transição de estados. Ao contrário da abordagem descendente, os analisadores *bottom-up* não impõem a restrição de ausência de recursão à esquerda, acomodando e processando com eficiência tanto a recursividade à direita quanto à esquerda. Ademais, são capazes de reconhecer virtualmente todas as construções sintáticas que podem ser descritas por gramáticas livres de contexto (AHO; SETHI; ULLMAN, 1995).

Entretanto, devido à enorme complexidade matemática requerida para a construção manual das tabelas de transição de estados LR, esses analisadores não costumam ser codificados à mão; ao invés disso, recorre-se a geradores automáticos de analisadores sintáticos, como a clássica ferramenta YACC, que produzem o código-fonte do *parser* a partir de uma especificação gramatical (COOPER; TORCZON, 2012).

2.1.5 Análise Semântica e Verificação de Tipos

Esta seção apresenta os fundamentos da análise semântica e da verificação de tipos, consolidando os conceitos e definições propostos por Aho, Sethi e Ullman (1995), Cooper e Torczon (2012), Price e Toscani (2001) e Nystrom (2021).

Uma vez que a estrutura gramatical do código é validada pela análise sintática, o processo de compilação evolui para a fase semântica. O objetivo central aqui é garantir que as construções do programa, embora sintaticamente corretas, possuam coerência lógica para a futura execução. Enquanto o *parser* se limita a verificar se a "forma" do código respeita as regras da linguagem, o analisador semântico atua como um inspetor que examina o significado das combinações entre os elementos do programa. Um dos pilares desse estágio é a resolução de nomes (ou amarrações), procedimento no qual o compilador mapeia cada identificador à sua declaração correspondente, respeitando as regras de escopo (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

A verificação de tipos constitui outro componente vital da análise semântica. Utilizando a

árvore de derivação e as informações armazenadas na tabela de símbolos, o compilador checa se os operadores estão sendo aplicados a operandos compatíveis. Durante essa inspeção, o sistema infere os tipos das expressões e detecta interpretações de valores que seriam semanticamente inválidas. Caso a linguagem permita, o analisador também pode aplicar coerções, que são conversões implícitas de tipos (como transformar um inteiro em ponto flutuante) para viabilizar operações entre tipos distintos (PRICE; TOSCANI, 2001; NYSTROM, 2021).

Tipos de Erros Semânticos

Diferente dos erros de sintaxe, as falhas semânticas ocorrem em códigos que possuem uma estrutura gramatical perfeita, mas que violam as regras de contexto ou significado da linguagem. O foco do tratamento de erros nesta fase é capturar inconsistências lógicas, destacando-se:

- **Incompatibilidade de Tipos:** Manifesta-se quando um operador é utilizado com operandos cujos tipos não podem interagir entre si, seja por falta de suporte da linguagem ou por ausência de regras de conversão. Um caso típico em linguagens fortemente tipadas é a tentativa de realizar operações aritméticas entre textos (*strings*) e valores lógicos (booleanos).
- **Identificadores Não Declarados e Violações de Escopo:** Este erro surge quando o programa tenta acessar uma variável, função ou objeto que não foi definido ou que não está visível no bloco de código atual. Um exemplo comum é a tentativa de utilizar uma variável local fora dos limites do seu escopo de definição.
- **Inconsistências de Aridade e Estruturais:** Refere-se a falhas na conformidade numérica de elementos. Isso inclui chamadas de funções com um número de parâmetros diferente do que foi estabelecido na assinatura original, ou o acesso a um *array* utilizando um número de dimensões que não condiz com sua declaração (AHO; SETHI; ULLMAN, 1995).

2.1.6 Geração de Código Intermediário

Os conceitos e as técnicas referentes à geração de código intermediário e à tradução dirigida pela sintaxe, detalhados nesta seção, fundamentam-se nas obras de Cooper e Torczon (2012), Nystrom (2021) e Price e Toscani (2001).

A geração de código intermediário atua como um elo estrutural entre as fases de análise (*front-end*) e síntese (*back-end*) de um compilador. Após a verificação estrutural e semântica do código fonte, o compilador normalmente gera uma representação intermediária explícita, cujo formato é projetado para ser largamente independente das especificidades da linguagem fonte e da arquitetura da máquina alvo. A principal vantagem dessa estratégia em múltiplos passos é resolver, de maneira gradativa, a barreira de complexidade existente na conversão direta de comandos de alto nível para instruções de baixo nível. Ademais, essa etapa viabiliza a otimização de código e facilita a portabilidade, permitindo que a mesma representação interna seja traduzida para diversas arquiteturas diferentes (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

Dentre as formas de representação linear utilizadas na literatura, destaca-se o uso do código de três endereços, que se assemelha a uma linguagem de montagem (*assembly*) genérica e abstrata. Nesta notação, as expressões matemáticas e lógicas são desmembradas em instruções elementares que possuem, no máximo, três operandos. A grande distinção teórica entre o código intermediário e o código objeto final reside no fato de que o intermediário se abstém de especificar detalhes físicos de *hardware*, como os registradores exatos e as posições absolutas de memória a serem alocadas (NYSTROM, 2021).

Para ilustrar o mecanismo, uma instrução aritmética convencional de atribuição, tal como $A := X + Y * Z$, seria mapeada em instruções *pseudo-assembly* mais simples, por meio da criação automática de variáveis temporárias pelo compilador. O fluxo de código de três endereços resultante desta operação seria composto por passos sequenciais lógicos:

1. $T1 := Y * Z$

2. $T2 := X + T1$

3. $A := T2$

Método de Geração Dirigida pela Sintaxe

Para que essa conversão ocorra de forma automatizada e correta, os compiladores comumente empregam a técnica denominada **tradução dirigida pela sintaxe**. Esse método estende os conceitos matemáticos das gramáticas livres de contexto, permitindo a associação direta de atributos aos símbolos gramaticais e a vinculação de ações semânticas específicas às regras de produção.

Na prática, isso significa que a geração de código pode ocorrer concomitantemente com a fase de análise sintática. À medida que o analisador (*parser*) agrupa as unidades léxicas e reconhece uma construção gramatical válida, ele executa em paralelo a ação semântica correspondente a essa regra. Essa execução processa os atributos passados pelos nós filhos da árvore gramatical e sintetiza as informações, permitindo a emissão gradativa e ordenada dos trechos de código intermediário (COOPER; TORCZON, 2012).

2.1.7 Otimização de Código e Eficiência Computacional

As discussões e definições apresentadas nesta seção fundamentam-se nas obras de Aho, Sethi e Ullman (1995), Cooper e Torczon (2012), Price e Toscani (2001) e Nystrom (2021).

Situada geralmente como a etapa intermediária do processo de compilação, a otimização visa refinar a representação interna do programa para torná-la mais performática. Esse aprimoramento busca não apenas acelerar a execução e reduzir o consumo de memória, mas também, em contextos tecnológicos modernos, diminuir o gasto energético do hardware. Durante esse estágio, o compilador substitui instruções genéricas por variantes mais ágeis e semanticamente equivalentes. Um exemplo clássico é o dobramento de constantes (*constant folding*), que antecipa cálculos de valores fixos para o tempo de compilação, evitando que sejam processados repetidamente durante a execução (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Para que uma técnica de otimização seja considerada válida, ela deve respeitar dois pilares fundamentais: a integridade (segurança) e a utilidade (rentabilidade). O princípio da segurança exige que qualquer modificação preserve o comportamento original do algoritmo, garantindo que o resultado final do programa não seja alterado. Já o princípio da rentabilidade assegura que a mudança estrutural resulte em um ganho de desempenho real e mensurável, justificando o tempo adicional despendido pelo compilador para realizar tal transformação (PRICE; TOSCANI, 2001).

Quanto à sua abrangência, o processo de otimização opera em diferentes níveis de profundidade, sendo categorizado em quatro escopos principais:

- **Otimização Local:** Concentra-se na análise de blocos básicos, que são sequências de instruções sem desvios internos. Nesse nível, é comum o uso de Grafos Acíclicos Dirigidos (GADs) para identificar e remover subexpressões redundantes e otimizar o uso de variáveis

locais.

- **Otimização Regional:** Estende a análise para grupos de blocos básicos, focando prioritariamente em laços de repetição (*loops*). Como a maior parte do tempo de execução de um software costuma ocorrer dentro desses ciclos, melhorias aplicadas aqui geram impactos significativos no desempenho global.
- **Otimização Global (Intraprocedural):** Examina o contexto completo de uma função ou procedimento. Através de análises detalhadas de fluxo de dados, o compilador consegue identificar oportunidades de melhoria que não seriam visíveis em uma inspeção puramente local.
- **Otimização Interprocedural:** Atua de forma macro, atravessando as fronteiras entre os diferentes módulos e funções do programa. Isso possibilita técnicas agressivas como a substituição em linha (*inlining*), onde o corpo de uma função é inserido diretamente no local da chamada, eliminando os custos computacionais associados ao desvio de execução e troca de contexto (COOPER; TORCZON, 2012).

De maneira geral, a maioria das rotinas de otimização ocorre em duas fases complementares: primeiro, uma análise estática para mapear o fluxo de controle e dependências; segundo, a transformação sintática que efetivamente reescreve o código. Entre as manobras mais frequentes, destacam-se a eliminação de código morto (instruções inacessíveis), o deslocamento de cálculos para fora de laços e a simplificação de redundâncias aritméticas (AHO; SETHI; ULLMAN, 1995).

2.1.8 Geração de Código Objeto e Mapeamento de Arquitetura

Os conceitos, desafios e estratégias referentes à fase de geração de código objeto e mapeamento de arquitetura, descritos nesta seção, fundamentam-se nas obras de Aho, Sethi e Ullman (1995), Cooper e Torczon (2012), Nystrom (2021) e Price e Toscani (2001).

A fase de geração de código objeto (ou geração de código de máquina) constitui a etapa final do processo de compilação, englobando as atividades finais do *back-end*. O principal objetivo desta fase é mapear a representação intermediária otimizada para a linguagem nativa da máquina alvo, garantindo que o código gerado seja logicamente correto, faça uso efetivo dos recursos de *hardware* disponíveis e execute de forma altamente eficiente (AHO; SETHI; ULLMAN, 1995).

Um dos maiores desafios enfrentados na geração de código objeto advém da diversidade arquitetônica dos processadores. Cada família de processadores (como as arquiteturas x86, ARM ou SPARC) dispõe de um conjunto próprio de instruções de máquina, também conhecido como Arquitetura do Conjunto de Instruções (ISA - *Instruction Set Architecture*). Isso implica que as operações de baixo nível e suas respectivas codificações binárias diferem radicalmente de um computador para outro (COOPER; TORCZON, 2012).

Para transpor essa barreira e evitar a necessidade de se construir um compilador inteiramente novo para cada arquitetura de *hardware* existente, os compiladores modernos adotam o princípio da *retargetabilidade* (capacidade de redirecionamento). Essa estratégia é viabilizada pelo uso do código intermediário: o *front-end* atua de forma independente de máquina para gerar a representação intermediária, permitindo que a adaptação para um novo computador exija apenas a reescrita ou a substituição do módulo gerador de código (*back-end*) (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

O processo de geração de código específico para uma máquina baseia-se fortemente em duas atividades interligadas: a seleção de instruções e a alocação de registradores. A seleção de instruções atua como um reconhecedor de padrões que analisa a árvore ou a sequência linear do código intermediário e seleciona a sequência exata de operações na ISA do processador capaz de implementar aquele comportamento com o menor custo de execução possível. Em paralelo, a alocação de registradores gerencia o escasso conjunto de registradores físicos da Unidade Central de Processamento (UCP), decidindo quais valores em tempo de execução serão mantidos nestes componentes de altíssima velocidade e quais precisarão ser transferidos para a memória principal (PRICE; TOSCANI, 2001).

2.1.9 Engenharia de Software

A discussão a seguir, que detalha a dinâmica de execução do código de máquina e o papel do sistema operacional e do *hardware* nesse processo, fundamenta-se nas obras de Cooper e Torczon (2012), Nystrom (2021) e Price e Toscani (2001).

Após o processo de síntese do compilador, o programa objeto resultante é composto por uma sequência densa e linear de operações elementares codificadas diretamente em formato binário. Diferentemente da representação hierárquica e abstrata manipulada nas fases iniciais da compilação, como a árvore de derivação, o código de máquina não possui estruturas de alto nível; ele é estritamente de baixo nível, composto por instruções rudimentares que ocupam poucos

bytes e determinam ações diretas no *hardware*, como a movimentação de valores entre endereços de memória e registradores, ou a adição de inteiros (PRICE; TOSCANI, 2001).

Para que esse código binário seja efetivamente processado, o programa precisa ser carregado na memória principal, momento em que um *software* carregador (*loader*) e o sistema operacional preparam o ambiente de tempo de execução, dividindo a memória em segmentos específicos, como a área do código estruturado e as áreas de dados. A partir desse ponto, a execução do algoritmo é ditada pela Unidade Central de Processamento (UCP), que utiliza um registrador especial da arquitetura, frequentemente chamado de Contador de Programa (*Program Counter* - PC) ou Ponteiro de Instrução (*Instruction Pointer* - IP), para rastrear rigorosamente a próxima instrução a ser executada (COOPER; TORCZON, 2012).

A dinâmica de execução da máquina ocorre através de um ciclo mecânico e contínuo. O processador avança através da fita de instruções, decodificando o código de operação (*opcode*) de cada uma para compreender a semântica da ação requerida e, em seguida, executando-a em ordem. Nesse ambiente de baixo nível, o fluxo de controle, que em linguagens fontes é caracterizado por laços de repetição e desvios condicionais elaborados, é resolvido de maneira muito mais simples. A alteração do fluxo é realizada puramente por meio de operações de saltos (*jumps* ou *branches*), que escrevem um novo endereço de memória diretamente no Contador de Programa, desviando a execução de um ponto do código de máquina para outro (NYSTROM, 2021).

2.1.9.1 Gestão de Tarefas e Metodologias Ágeis

Para a fundamentação teórica acerca da gestão de escopo e dos princípios de desenvolvimento iterativo e incremental que orientaram este trabalho, esta subseção baseia-se nos princípios discutidos por AdaptWorks (2012) e toma como referência as diretrizes do *framework* Scrum detalhadas por Sabbagh (2014), das quais foram incorporadas práticas pontuais, como a organização do trabalho em ciclos curtos, sem a adoção de seus papéis e cerimônias formais.

O gerenciamento clássico de projetos de software, frequentemente baseado no modelo em cascata (*waterfall*), assume que o desenvolvimento pode ser tratado como uma sequência previsível e linear de atividades. No entanto, a engenharia de software lida com sistemas intrinsecamente complexos e com alto grau de incerteza, onde o engessamento de requisitos em fases iniciais costuma resultar em enorme desperdício e na entrega de produtos que não

atendem às reais necessidades do cliente. Em contraponto a essa visão, as metodologias ágeis assumem a imprevisibilidade natural do desenvolvimento, baseando-se no empirismo e em ciclos de *feedback* constante para promover a adaptação contínua e a entrega contínua de valor (PRESSMAN; MAXIM, 2016; SOMMERVILLE, 2018).

Neste contexto de ambientes complexos e mutáveis, o Scrum destaca-se como um *framework* ágil, simples e focado na eficácia. Em vez de depender de cronogramas rígidos de longo prazo (como os tradicionais gráficos de Gantt), a gestão de tarefas ocorre de forma empírica e adaptativa. Todo o trabalho a ser realizado no produto é centralizado e organizado em uma lista dinâmica, ordenada e emergente, denominada *Product Backlog*. Esta estrutura substitui os extensos e estáticos documentos de requisitos tradicionais, garantindo que o escopo seja gradualmente detalhado apenas com o nível de informação estritamente necessário para o momento da execução, evitando documentações prematuras (SABBAGH, 2014).

Para viabilizar essa execução com excelência, o desenvolvimento é fatiado em ciclos curtos e de duração fixa, chamados de *Sprints*. A cada início de ciclo, as tarefas mais prioritárias são extraídas do topo do *Product Backlog* e detalhadas de forma atômica no *Sprint Backlog*. Essa divisão iterativa e atômica do trabalho permite que uma equipe multidisciplinar e auto-organizada foque exclusivamente em metas de negócio de curto prazo. Consequentemente, o projeto avança passo a passo, mitigando os riscos tecnológicos, abraçando a mudança como uma vantagem competitiva e permitindo a reavaliação contínua das prioridades do software (ADAPTWORKS, 2012; SABBAGH, 2014).

2.1.9.2 Testes Automatizados e *Test-Driven Development* (TDD)

Para a fundamentação da estratégia de testes e da adoção do *Test-Driven Development* (TDD) descritas nesta subseção, este trabalho baseia-se nos princípios de qualidade e profissionalismo de Martin (2011), na estruturação arquitetural dos testes defendida por Martin (2019), e na literatura específica de testes e *design* de software de Aniche (2012) e Silveira et al. (2012).

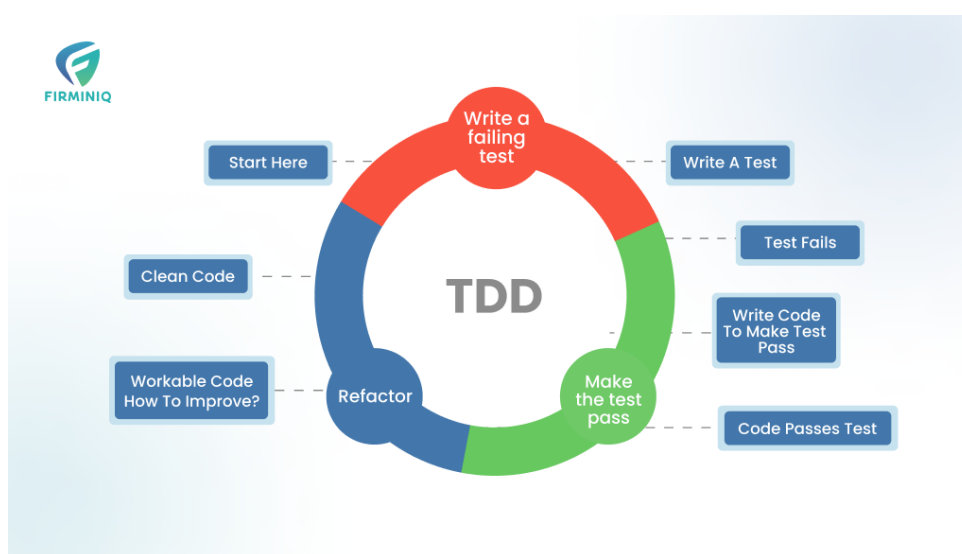
A garantia da qualidade de um software exige a validação contínua de seus comportamentos. Em projetos modernos, a automação de testes substitui a exaustiva e falha verificação manual, fornecendo uma rede de segurança que permite aos desenvolvedores alterar, refatorar e evoluir o código com a certeza de que as funcionalidades preexistentes não foram comprometidas. Essa prevenção contra a quebra de comportamentos já consolidados é conhecida

como teste de regressão. Além de atestar o funcionamento da aplicação, os testes automatizados atuam como uma documentação viva e executável das regras de negócio do sistema (ANICHE, 2012).

Nesse contexto, destaca-se a prática do *Test-Driven Development* (Desenvolvimento Guiado por Testes). O TDD inverte o ciclo tradicional de engenharia, exigindo que o desenvolvedor escreva um teste automatizado antes mesmo de implementar a funcionalidade do sistema (ANICHE, 2012; MARTIN, 2011).

O processo segue um ciclo de três etapas fundamentais, muitas vezes chamado de ciclo vermelho-verde-refatora (*Red-Green-Refactor*), conforme ilustrado na Figura 2.3. Primeiro, escreve-se um teste que falha, definindo o comportamento desejado; em seguida, escreve-se o código mais simples possível para fazer o teste passar; por fim, o código de produção é refatorado para remover duplicações e melhorar sua estrutura.

Figura 2.3 – Ciclo de Desenvolvimento Guiado por Testes (TDD).



Fonte: Firminiq (2024).

Mais do que uma simples técnica de verificação, o TDD é uma poderosa ferramenta de *design* de software. Ao escrever o teste primeiro, o desenvolvedor atua como o primeiro cliente do próprio código, o que o obriga a pensar na interface pública da classe, em suas dependências e em sua responsabilidade. Dificuldades na escrita de um teste, como a necessidade de configurações complexas ou o uso excessivo de dependências simuladas (*mock objects*), são indícios claros de problemas arquiteturais, denunciando alto acoplamento e baixa coesão estrutural (MARTIN, 2019; ANICHE, 2012).

Para maximizar a eficiência de execução e manutenção, a estratégia de testes de um

sistema deve ser distribuída em diferentes níveis de granularidade, conceito frequentemente ilustrado pela Pirâmide de Testes. A base da pirâmide é composta pelos testes de unidade, que são extremamente rápidos, isolados e verificam o comportamento de pequenas partes do código, como classes ou métodos específicos. O nível intermediário abriga os testes de integração, responsáveis por garantir a correta comunicação entre os componentes internos e sistemas de infraestrutura externos, como bancos de dados ou serviços de terceiros. No topo da pirâmide estão os testes de ponta a ponta (*end-to-end*) e de aceitação, que validam o sistema inteiro através da interface com o usuário, simulando o comportamento real da aplicação de forma mais custosa e lenta (ANICHE, 2012; SILVEIRA et al., 2012).

Por fim, do ponto de vista arquitetural, os testes devem ser tratados como componentes essenciais do sistema. Contudo, para que não se tornem frágeis e difíceis de manter, eles devem ser desacoplados de elementos voláteis, como a Interface de Usuário (UI) ou as ferramentas de persistência. Uma arquitetura limpa e bem projetada permite que as regras de negócio centrais sejam exaustivamente testadas de forma isolada, garantindo execuções rápidas e *feedback* imediato para a equipe de desenvolvimento (MARTIN, 2019).

2.1.9.3 Arquitetura Limpa e Separação de Responsabilidades

Para a fundamentação das decisões arquiteturais e dos padrões de projeto adotados neste trabalho, esta subseção baseia-se primordialmente nos preceitos de Arquitetura Limpa propostos por Martin (2019) e nas diretrizes de design de software estruturadas por Silveira et al. (2012).

O projeto arquitetural do sistema foi concebido sob a ótica da Separação de Responsabilidades (*Separation of Concerns* - SoC), dividindo o software em camadas isoladas com papéis independentes e bem definidos. A interface de usuário no *front-end* foca exclusivamente na experiência de uso e integração visual, enquanto a lógica de execução do negócio permanece no *back-end*. Isso respeita a premissa fundamental da Arquitetura Limpa de que as regras centrais da aplicação jamais devem depender de detalhes periféricos de entrega, considerando que a tecnologia *Web* ou a interface de usuário são apenas detalhes de implementação (MARTIN, 2019).

No que tange à camada de dados, a persistência foi totalmente abstraída por meio de ferramentas de Mapeamento Objeto-Relacional (ORMs, como *Prisma* e *SQLAlchemy*). O uso de ORMs atua como uma ponte que suaviza a discrepância entre o paradigma relacional e o orientado a objetos, mitigando a dependência direta de consultas SQL. Essa prática garante que

o banco de dados seja tratado estritamente como um detalhe de infraestrutura de baixo nível, impedindo que sua estrutura tabular polua a lógica central e as regras de negócio do domínio (MARTIN, 2019; SILVEIRA et al., 2012).

Por fim, a maturidade da arquitetura expressa-se também pela adoção de Padrões de Projeto (*Design Patterns*) na construção do compilador. O uso do padrão *Factory*, por exemplo, foi empregado para encapsular a lógica complexa de instanciação dos componentes de análise (*Lexer* e *Scanners*). Essa abordagem centraliza a decisão de criação de objetos, favorecendo a coesão e reduzindo drasticamente o acoplamento global do sistema (SILVEIRA et al., 2012).

2.1.9.4 Qualidade de Código e *Clean Code*

Para o detalhamento conceitual e a definição das práticas de *Clean Code* (Código Limpo) descritas nesta subseção, este trabalho fundamenta-se nas consolidadas diretrizes de Martin (2009) e na literatura de apoio sobre metodologias ágeis abordada por AdaptWorks (2012).

A construção de um software ágil e sustentável esbarra invariavelmente na qualidade de sua base de código. O código-fonte é a linguagem formal na qual os requisitos de um sistema são expressos e detalhados de modo que a máquina possa executá-los. No entanto, sob a pressão de prazos, é comum que as equipes produzam códigos desorganizados, resultando em uma degradação estrutural contínua que, a longo prazo, faz a produtividade despencar, podendo até mesmo inviabilizar a evolução do projeto (MARTIN, 2009).

Para combater essa entropia, a adoção das práticas de Código Limpo torna-se essencial. A escrita de um código limpo exige disciplina, o uso consciente de padrões e uma sensibilidade aguçada para reconhecer a desorganização e aplicar refatorações. Um código limpo deve ser elegante, simples, direto e, sobretudo, legível para os seres humanos. O profissionalismo no desenvolvimento de software pressupõe que o código não deve apenas funcionar, mas deve ser fácil de entender, alterar e estender por outros desenvolvedores (MARTIN, 2009).

Dentre as diretrizes estruturais básicas e inegociáveis para a manutenção dessa clareza na base de código, destacam-se:

- **Nomes Significativos:** A nomenclatura de variáveis, funções, parâmetros e classes deve revelar claramente a sua intenção e responder ao porquê de sua existência e ao que o componente faz. Deve-se evitar o uso de siglas obscuras, codificações desnecessárias ou trocadilhos que exijam mapeamentos mentais, facilitando assim a leitura fluida e a busca

no código.

- **Funções Pequenas e Focadas:** Uma função deve ser extremamente concisa e projetada para fazer estritamente uma única coisa, mantendo um único nível de abstração. Funções extensas que realizam múltiplas tarefas, agrupam várias seções lógicas ou que misturam tratamento de erros com a lógica de negócio violam a organização e dificultam o entendimento e a testabilidade.
- **Uso Consciente de Comentários:** O uso de comentários costuma compensar a incapacidade do programador de se expressar claramente através do próprio código. Eles não devem ser utilizados para encobrir ou justificar um código ruim ou confuso. A energia da equipe deve ser direcionada para tornar o código tão claro e autoexplicativo que elimine a necessidade de anotações redundantes.
- **Eliminação de Duplicidade e Refatoração contínua:** A repetição de código é o grande inimigo de um sistema bem desenvolvido, pois representa trabalho, risco e complexidade desnecessária. Deve-se melhorar o design do código de forma contínua, aplicando refatorações que tornem o sistema mais enxuto e compreensível, sem alterar o seu comportamento externo (MARTIN, 2009).

2.1.9.5 FastAPI: Framework para APIs de Alto Desempenho

Os conceitos, as características e os fundamentos arquiteturais referentes ao *framework* FastAPI, detalhados nesta subseção, fundamentam-se na documentação oficial do FASTAPI (2026), nas análises comparativas de DASROOT (2026) e no material didático de DUNOSSAURO (2026).

O FastAPI é um *framework* web moderno e de alto desempenho projetado para a construção de Interfaces de Programação de Aplicações (APIs) utilizando a linguagem Python. Sua arquitetura foi desenhada para ser simples, rápida e eficiente, tirando proveito das funcionalidades modernas da linguagem, como as anotações de tipo explícitas (*type hints*) e o suporte nativo a operações da programação assíncrona (FASTAPI, 2026).

Um dos pilares que garantem a alta performance do FastAPI é a sua fundação sobre a especificação ASGI (*Asynchronous Server Gateway Interface*) e o *framework* Starlette. Diferentemente de *frameworks* web tradicionais que utilizam o modelo síncrono (WSGI), a arquitetura assíncrona do FastAPI permite que o servidor gerencie milhares de conexões

simultâneas de forma eficiente, processando operações de entrada e saída (I/O) sem bloquear a *thread* de execução. Em ambientes de produção, aliado a servidores de aplicação como o Uvicorn, o FastAPI é capaz de suportar picos de mais de 20.000 requisições por segundo, apresentando desempenho comparável a linguagens compiladas como Go e plataformas como o NodeJS (DASROOT, 2026).

Outra característica central do *framework* é a integração profunda com a biblioteca Pydantic, que assume a responsabilidade pela validação e serialização rigorosa dos dados. Ao invés de exigir a aprendizagem de uma nova micro-linguagem de validação, o desenvolvedor utiliza os tipos de dados padrão do próprio Python. A ferramenta converte automaticamente os dados de entrada (como o corpo de uma requisição em formato JSON) para os tipos corretos e gera mensagens de erro estruturadas e detalhadas caso a validação falhe, garantindo uma maior segurança e consistência no processamento (FASTAPI, 2026; DUNOSSAURO, 2026).

Além disso, a plataforma é construída estritamente em torno de padrões abertos, notadamente a especificação OpenAPI e o JSON Schema. Essa aderência aos padrões permite que o FastAPI gere de forma automática e dinâmica a documentação interativa da API, disponibilizando interfaces prontas para o consumo do desenvolvedor, como o Swagger UI e o ReDoc. Isso reduz drasticamente o tempo gasto com a documentação manual, facilita a integração com sistemas de *front-end* e permite a geração automática de clientes (*SDKs*) em diversas linguagens (FASTAPI, 2026).

Por fim, o *framework* incorpora um sistema de Injeção de Dependência poderoso, porém intuitivo, que simplifica a integração do código da aplicação com bancos de dados relacionais (através de ORMs como o SQLAlchemy), além de prover mecanismos nativos e facilitados para a implementação de segurança e autenticação, como fluxos OAuth2 e *tokens* JWT. Toda essa gama de funcionalidades faz do FastAPI uma escolha robusta, tanto para a criação rápida de protótipos quanto para sistemas corporativos altamente escaláveis (DUNOSSAURO, 2026).

2.2 Softwares Educativos e Tecnologias de Apoio à Aprendizagem

Para a fundamentação conceitual acerca do uso de tecnologias no ambiente escolar abordada nesta subseção, este trabalho apoia-se nas perspectivas sobre plataformas educativas delineadas por Benedetti (2025) e nas diretrizes de inovação em tecnologia educacional da Mind Makers (2025).

Uma plataforma educativa é um ambiente virtual desenvolvido para apoiar processos de ensino e aprendizagem, oferecendo recursos, ferramentas e conteúdos que facilitam a interação entre alunos, professores e instituições de ensino. Nesse cenário de transformação, a escolha da ferramenta certa não é apenas uma questão tecnológica, mas um fator determinante para o engajamento, a motivação e o sucesso educacional (BENEDETTI, 2025).

Contudo, para aproveitar o potencial das novas tecnologias, a instituição de ensino precisa ir além da simples adoção de ferramentas digitais. É essencial entender o propósito pedagógico por trás de cada inovação, garantindo que a tecnologia esteja sempre a serviço da aprendizagem e não o contrário. Afinal, a tecnologia educacional atua como um meio que exige professores preparados para usar recursos digitais de forma criativa e estratégica, conectando os alunos a um aprendizado relevante, dinâmico e significativo (Mind Makers, 2025).

Quando projetada com esse foco pedagógico, uma plataforma educativa bem estruturada permite a personalização do aprendizado, ajustando-se ao ritmo e ao estilo de cada aluno, o que maximiza seu potencial e promove uma assimilação muito mais eficaz do conteúdo. Essa abordagem valoriza o tempo de cada estudante e amplia as suas capacidades ao longo de toda a jornada educacional. Além disso, a integração de dinâmicas interativas nas ferramentas de ensino, como a gamificação e a aprendizagem baseada em desafios, transforma a educação em um processo envolvente de recompensas. Esses sistemas lúdicos ajudam a despertar o interesse contínuo dos alunos e a desenvolver habilidades socioemocionais indispensáveis, como persistência e colaboração (BENEDETTI, 2025; Mind Makers, 2025).

2.3 Trabalhos Relacionados

Esta seção analisa ferramentas e linguagens que compartilham objetivos com a solução proposta, seja no apoio ao ensino de programação, seja na construção de compiladores para fins didáticos. O critério de seleção dos trabalhos aqui analisados, baseado na representatividade dos dois eixos do problema, está descrito na Seção 4.1. Cada trabalho é descrito quanto à sua proposta, às suas características e às suas limitações; ao final, uma análise crítica consolida a comparação e evidencia o diferencial da plataforma desenvolvida frente ao estado da arte.

2.3.1 Portugol Studio e Portugol Webstudio

A fundamentação teórica e as características referentes ao ambiente Portugol Studio e sua versão web, discutidas nesta subseção, baseiam-se nas publicações de GOMES (2020),

MARTINS (2025) e nos materiais oficiais da UNIVALI (2026).

O Portugol Studio é um Ambiente de Desenvolvimento Integrado (IDE) educacional de código aberto, projetado e mantido pelo Laboratório de Inovação Tecnológica na Educação (LITE) da UNIVALI. Seu principal objetivo é facilitar o aprendizado da lógica de programação para iniciantes, abstraindo a complexidade de ambientes profissionais. A ferramenta permite que os alunos escrevam algoritmos utilizando o Portugol, uma linguagem com sintaxe baseada em C e PHP, porém com comandos totalmente estruturados na língua portuguesa (UNIVALI, 2026).

Para auxiliar a compreensão do comportamento da máquina por parte do estudante, o software oferece recursos didáticos visuais fundamentais, destacando-se a execução passo a passo (depurador), um inspetor de variáveis em tempo real e uma árvore estrutural do código. Ademais, visando a acessibilidade e a redução de barreiras tecnológicas, o projeto evoluiu para o Portugol Webstudio, uma versão que permite a compilação e a execução dos códigos diretamente no navegador, dispensando qualquer necessidade de instalação no computador do usuário. Outro ponto de destaque é o suporte nativo a bibliotecas gráficas e de som, o que possibilita a criação de jogos completos com interface gráfica e aumenta significativamente o engajamento dos estudantes (GOMES, 2020; MARTINS, 2025).

2.3.2 Laila: Gerador de Analisadores Léxicos e Sintáticos Online

A descrição arquitetural, as funcionalidades e os resultados da aplicação da plataforma web Laila, discutidos nesta subseção, fundamentam-se inteiramente na pesquisa desenvolvida por Silva (2021).

A plataforma Laila foi concebida para mitigar as dificuldades práticas enfrentadas por estudantes na disciplina de Compiladores. Tradicionalmente, o ensino prático dessa matéria exige a instalação e configuração de ferramentas complexas, como o GCC (coleção de compiladores C), o FLEX e o Bison, tarefas que costumam ser árduas em sistemas operacionais não baseados em Linux. Para solucionar esse gargalo, a ferramenta oferece um ambiente de desenvolvimento integrado (IDE) totalmente online, permitindo que o aluno projete, escreva e teste seus analisadores léxicos e sintáticos sem a necessidade de configurações locais (SILVA, 2021).

A arquitetura do sistema foi desenvolvida utilizando tecnologias web modernas. O módulo de interação com o usuário (*front-end*) foi construído em JavaScript com o *framework* VueJS e a biblioteca CodeMirror, garantindo uma edição de código com destaque de sintaxe e

identificação de erros em tempo real. A comunicação e o processamento ocorrem por meio de uma API REST desenvolvida em Python (*back-end*), que recebe os códigos do usuário, executa as ferramentas de geração FLEX e Bison no servidor, captura eventuais falhas (como laços infinitos ou erros de compilação) e retorna as saídas diretamente para o navegador do estudante (SILVA, 2021).

Em estudos de caso realizados com turmas de Ciência da Computação, a ferramenta demonstrou resultados altamente positivos. O uso da plataforma eliminou os problemas de compatibilidade e engajou os estudantes, que relataram uma melhoria significativa na compreensão dos algoritmos de análise léxica e sintática devido à praticidade de testar as linguagens em tempo real (SILVA, 2021).

2.3.3 Quorum: Linguagem de Programação Baseada em Evidências

Os conceitos, os estudos empíricos de sintaxe e as características arquiteturais da linguagem Quorum, detalhados nesta subseção, fundamentam-se nas pesquisas de Stefik e Siebert (2013), nos relatos de Ladner (2019) e na documentação oficial do projeto QUORUM (2025).

A linguagem Quorum foi inicialmente concebida pelo pesquisador Andreas Stefik com o objetivo de ser nativamente acessível para estudantes cegos ou com deficiência visual. Para atingir esse objetivo, a linguagem foi projetada para ter uma leitura amigável e clara em leitores de tela. Com o passar do tempo e o ganho de popularidade, o projeto evoluiu para uma linguagem de propósito geral, mantendo sua premissa principal: ser a primeira linguagem de programação orientada a evidências. Isso significa que suas regras sintáticas e semânticas não foram escolhidas com base em tradição ou intuição, mas sim através de testes rigorosos e ensaios clínicos randomizados com usuários novatos e experientes (STEFIK; SIEBERT, 2013).

Durante a concepção da linguagem, estudos empíricos demonstraram que a sintaxe tradicional baseada na linguagem C (amplamente adotada pelo Java, Perl e outras) atua como uma barreira significativa para estudantes iniciantes. Os experimentos utilizaram métricas como o Mapa de Precisão de Tokens e compararam o desempenho de novatos utilizando linguagens comerciais contra uma linguagem "placebo", cujas palavras-chave foram geradas aleatoriamente. Os resultados comprovaram que a remoção de símbolos obscuros (como chaves, colchetes e pontos e vírgulas) e a substituição por palavras de uso comum reduzem drasticamente as taxas de erro sintático. Como resultado prático, no Quorum, em vez de se utilizar um laço de repetição

complexo, o programador escreve comandos diretos como "repeat 10 times", o que diminui a carga cognitiva necessária para a memorização da estrutura (STEFIK; SIEBERT, 2013).

Para suportar o desenvolvimento, a equipe do projeto criou o Quorum Studio, um Ambiente de Desenvolvimento Integrado (IDE) nativamente acessível que permite a edição, compilação e depuração de códigos. A infraestrutura do Quorum é flexível, permitindo que a linguagem seja executada offline, gerando bytecodes para a Máquina Virtual Java (JVM), ou online diretamente no navegador do usuário, suportando desde lógicas simples até a criação de jogos, processamento de áudio e robótica (LADNER, 2019; QUORUM, 2025).

2.3.4 chibicc: Uma Abordagem Incremental para o Ensino de Compiladores

As características arquiteturais e a metodologia educacional do compilador *chibicc*, abordadas nesta subseção, fundamentam-se no projeto e na documentação de Ueyama (2020).

O *chibicc* é um compilador C de pequeno porte, desenvolvido especificamente como uma implementação de referência para o ensino de construção de compiladores e programação de baixo nível. Apesar de ser categorizado como um compilador educacional, ele suporta uma ampla variedade de recursos da especificação C11, sendo capaz de compilar corretamente programas de grande porte do mundo real, como o sistema de controle de versão Git e o banco de dados SQLite, sem a necessidade de modificações no código-fonte dessas aplicações (UEYAMA, 2020).

O grande diferencial pedagógico dessa ferramenta reside na sua abordagem de ensino incremental. Em vez de apresentar ao estudante uma arquitetura monolítica e complexa desde o início, o projeto orienta a construção do compilador passo a passo. O aluno começa desenvolvendo um tradutor rudimentar capaz de aceitar e processar apenas um único número como linguagem válida. A partir dessa base mínima, novos recursos gramaticais e semânticos são adicionados de forma isolada em cada etapa do estudo, até que a linguagem aceita pelo compilador corresponda à especificação completa do C11. Esse método construtivo desmistifica a transição do código-fonte para o código de máquina e oferece uma compreensão profunda sobre o funcionamento interno das linguagens (UEYAMA, 2020).

2.3.5 Beecrowd: Prática de Algoritmos e Avaliação Automatizada

As análises sobre a plataforma Beecrowd, suas características de avaliação automatizada e seu impacto como tecnologia de informação e comunicação no ensino de algoritmos, discutidas

nesta subseção, fundamentam-se no estudo de caso de Ferreira, Silva e Ribeiro (2025), no artigo de Garcia (2024) e nos materiais de MÃO NO CÓDIGO (2022).

O Beecrowd, anteriormente conhecido como URI Online Judge, é uma plataforma online amplamente utilizada no meio acadêmico e em treinamentos para maratonas de programação competitiva. Seu acervo conta com milhares de desafios algorítmicos divididos por categorias e níveis de dificuldade, desde lógicas fundamentais até estruturas de dados complexas. A plataforma permite que estudantes e profissionais submetam soluções utilizando diversas linguagens de programação de mercado, como C++, Python, Java e JavaScript (GARCIA, 2024).

A principal característica da ferramenta é o seu sistema de juiz online (*Online Judge*). O fluxo de uso consiste em o aluno ler a especificação de um problema, desenvolver o código e submetê-lo na plataforma. Em questão de segundos, o sistema compila, executa e testa a solução contra diversos casos de teste fechados, fornecendo um *feedback* imediato, como “*Accepted*” para sucesso ou “*Wrong Answer*” e “*Presentation Error*” para falhas. Essa validação em tempo real, aliada a elementos de gamificação como fóruns e *rankings* (a chamada *Vibe Coding*), estimula a prática contínua, tirando o aluno de uma posição passiva e colocando-o ativamente na resolução de problemas (FERREIRA; SILVA; RIBEIRO, 2025; MÃO NO CÓDIGO, 2022).

2.3.6 Análise Crítica e o Diferencial da Solução Proposta

A análise comparativa e as conclusões desenvolvidas a seguir fundamentam-se no contraste entre a arquitetura proposta neste Trabalho de Conclusão de Curso e as ferramentas educacionais discutidas nas subseções anteriores, com base nas pesquisas de Silva (2021), Stefik e Siebert (2013), Ueyama (2020), Ferreira, Silva e Ribeiro (2025) e nos materiais da UNIVALI (2026).

Ao observar o estado da arte das ferramentas de apoio ao ensino de programação e de compiladores, nota-se que as soluções existentes tendem a seguir dois caminhos distintos, ambos com limitações estruturais significativas para o aluno iniciante. O primeiro caminho é o das ferramentas com foco em lógica e prática contínua, como o Portugol Studio, a linguagem Quorum e a plataforma Beecrowd. Embora o Quorum comprove cientificamente que a sintaxe tradicional é uma barreira real de aprendizado (STEFIK; SIEBERT, 2013) e o Portugol traduza os comandos para a língua nativa, todas essas plataformas compartilham um obstáculo comum: a imposição de uma gramática estática e rigorosa. O aluno continua obrigado a memorizar palavras-chave fixas e a se adequar a avaliadores automáticos inflexíveis, que rejeitam lógicas

corretas devido a erros mínimos de formatação ou digitação.

O segundo caminho engloba as ferramentas criadas especificamente para o ensino prático da disciplina de compiladores, como a plataforma Laila e o projeto *chibicc*. Apesar de serem soluções de excelência em seus respectivos nichos seja abstraindo configurações de ambiente por meio de plataformas web, ou ensinando a construção incremental de um tradutor, elas possuem um enfoque estritamente sistêmico e de baixo nível. São metodologias voltadas para alunos avançados de Ciência da Computação, exigindo o domínio prévio de linguagens complexas (como C11) e de ferramentas geradoras pesadas (como FLEX e Bison), o que as torna inadequadas para a quebra da barreira linguística introdutória.

É exatamente na intersecção e na superação dessas lacunas que o presente projeto se destaca de forma inovadora. O interpretador dinâmico construído neste trabalho desloca toda a complexidade arquitetural da análise léxica e sintática para os bastidores, oferecendo uma IDE web focada inteiramente na flexibilidade cognitiva do usuário iniciante. Em vez de impor uma "sintaxe ideal única" ou de focar no mapeamento de instruções para o *hardware*, a plataforma delega a criação estrutural da linguagem ao próprio estudante.

Ao permitir que o aluno personalize as palavras-chave, os operadores e as regras estruturais de sua própria linguagem de forma dinâmica no navegador, o sistema atua como um laboratório seguro de experimentação algorítmica. Essa abordagem suprime a necessidade de adaptação imediata a um idioma comercial fixo, mitigando drasticamente a carga cognitiva e a frustração atrelada aos erros gramaticais. Dessa forma, o software proposto consolida-se não apenas como um ambiente de execução, mas como uma ponte pedagógica: uma ferramenta que prioriza a estrutura do pensamento computacional e suaviza a curva de aprendizado, preparando o estudante com muito mais confiança antes de sua transição para as linguagens estritas e juízes online do mercado de software.

3 PROPOSTA

Este capítulo apresenta o escopo do trabalho e o cronograma de desenvolvimento das duas etapas do TCC. O projeto consiste em construir uma plataforma educacional web composta por uma IDE acoplada a um interpretador dinâmico. A plataforma permite ao usuário personalizar o vocabulário da linguagem, os delimitadores de bloco, os operadores em forma de palavra, os literais booleanos e as regras de finalização de instrução. O objetivo é reduzir a dificuldade inicial de aprender programação deslocando o foco do aluno da sintaxe para a lógica.

O desenvolvimento foi dividido em seis etapas principais, parcialmente sobrepostas: (I) revisão bibliográfica sobre interpretadores, compiladores e ferramentas educacionais; (II) definição dos requisitos e da arquitetura do sistema; (III) implementação da análise léxica e sintática com suporte a regras gramaticais configuráveis; (IV) geração e execução do código intermediário e verificação semântica; (V) integração entre o núcleo do interpretador e a plataforma web; e (VI) testes, validação com usuários e coleta de feedback. O cronograma de execução dessas etapas está na Tabela 3.1, onde cada coluna numerada representa um mês corrido e as células marcadas com “X” indicam os meses de execução previstos.

Tabela 3.1 – Cronograma de desenvolvimento do trabalho de conclusão de curso.

Atividade	TCC I						TCC II					
	01	02	03	04	05	06	07	08	09	10	11	12
Revisão Bibliográfica	X	X	X	X								
Escrita da Introdução e Revisão de Literatura		X	X	X								
Predefinição de escopo e Especificação dos comandos dinâmicos			X	X								
Escrita da Metodologia			X		X	X	X					
Arquitetura e Modelagem do Interpretador			X	X								
Implementação da Análise Léxica e Sintática				X	X							
Geração de Código Intermediário e Análise Semântica				X	X	X						
Integração entre a plataforma e interpretador					X	X	X					
Testes, Validação com Usuários e Coleta de <i>Feedback</i>									X	X	X	
Escrita do TCC (Resultados, Discussão e Conclusão)									X	X	X	X

Fonte: elaborado pelo autor.

Este trabalho foi desenvolvido em dupla, conforme a pré-proposta aprovada, que distribui o projeto em **funções complementares**: o desenvolvimento da plataforma web e da interface de

personalização da linguagem ficou a cargo de Igor Lamoia Queiroz, enquanto a concepção e a implementação do **núcleo do interpretador** (análise léxica, sintática e semântica, geração e execução de código intermediário) constituem a demanda desta monografia. As etapas iniciais de revisão bibliográfica e de definição da arquitetura geral foram conduzidas conjuntamente, e o regime de dupla manifestou-se sobretudo nas atividades de fronteira entre os subsistemas: a definição dos contratos de comunicação entre os módulos e a integração do núcleo à plataforma, em que decisões de uma camada exigiam ajustes na outra. O aprofundamento técnico e analítico do núcleo de tradução, objeto de avaliação deste documento, é de responsabilidade individual do autor.

A divisão de eixos de aprofundamento individual, prevista na pré-proposta, aparece nos capítulos de redação acadêmica. Neste documento, o foco está no núcleo de tradução: as fases de análise léxica, sintática e semântica, a geração de código intermediário e o funcionamento do interpretador. O aluno Igor Lamoia Queiroz aprofunda, em seu documento, a IDE web: a interface de personalização da linguagem, a usabilidade da plataforma e a coleta de feedback dos estudantes. Essa divisão se justifica pela amplitude do projeto, que seria difícil de cobrir com profundidade suficiente por apenas um autor.

Com o TCC I concluído, o TCC II dará continuidade ao aprofundamento do núcleo de tradução, ampliando a cobertura de testes e a avaliação técnica do interpretador (a correção do código intermediário gerado e a robustez frente às variações configuráveis da linguagem) e consolidando a análise comparativa entre a solução desenvolvida e as ferramentas discutidas no referencial teórico, sob a ótica das capacidades do compilador. Essa comparação evidencia a convergência da proposta com sistemas relacionados de compiladores, pois o projeto também explicita as etapas de análise léxica, análise sintática e geração de representação intermediária; contudo, diferencia-se ao integrar essas etapas a uma linguagem configurável e a uma IDE educacional voltada ao estudante iniciante. O próximo capítulo descreve a modelagem e a implementação realizadas até o fim do TCC I.

4 METODOLOGIA

Este capítulo descreve a abordagem metodológica adotada para o desenvolvimento do interpretador dinâmico proposto, detalhando a arquitetura geral do sistema, as decisões de projeto que sustentam cada fase do compilador implementado e os mecanismos de integração com a plataforma web. A condução do trabalho seguiu as etapas previstas no cronograma do projeto, contemplando, nesta primeira fase do Trabalho de Conclusão de Curso, a revisão bibliográfica, a especificação do escopo da linguagem e de seus elementos configuráveis, a modelagem da arquitetura do interpretador, a implementação das análises léxica e sintática, a geração e a execução do código intermediário e a integração entre o núcleo do compilador e o ambiente web.

A construção do sistema foi conduzida de forma incremental e iterativa, organizando o trabalho em ciclos curtos de implementação e validação. Não se adotou formalmente um *framework* de gestão ágil (cujos papéis e cerimônias pressupõem equipes maiores do que a dupla responsável por este projeto), mas sim os princípios de desenvolvimento iterativo e incremental discutidos por AdaptWorks (2012), combinados à prática de Desenvolvimento Guiado por Testes consolidada por Aniche (2012) e Martin (2011). Cada etapa do compilador foi modelada, implementada e validada por meio de testes automatizados antes da agregação à etapa seguinte. Essa abordagem permitiu a verificação contínua de regressões durante a evolução das gramáticas configuráveis e a manutenção da estabilidade das fases de análise mesmo diante das frequentes mudanças impostas pela natureza dinâmica da linguagem proposta.

4.1 Seleção dos Trabalhos Relacionados

A escolha dos trabalhos analisados na Seção 2.3 não seguiu uma revisão sistemática da literatura, mas um critério de representatividade: foram selecionadas ferramentas e linguagens que ilustram os dois eixos por meio dos quais a literatura aborda o problema tratado neste trabalho. O primeiro eixo reúne soluções voltadas ao apoio à lógica e à prática de programação para iniciantes, com ênfase na redução da barreira sintática, e é representado pelo Portugol Studio, pela linguagem Quorum e pela plataforma Beecrowd. O segundo eixo reúne ferramentas voltadas ao ensino da construção de compiladores e analisadores, e é representado pela plataforma Laila e pelo projeto *chibicc*.

Como critérios de inclusão, considerou-se que cada trabalho (i) tivesse propósito educacional explícito no ensino de programação ou de compiladores; (ii) fosse acessível

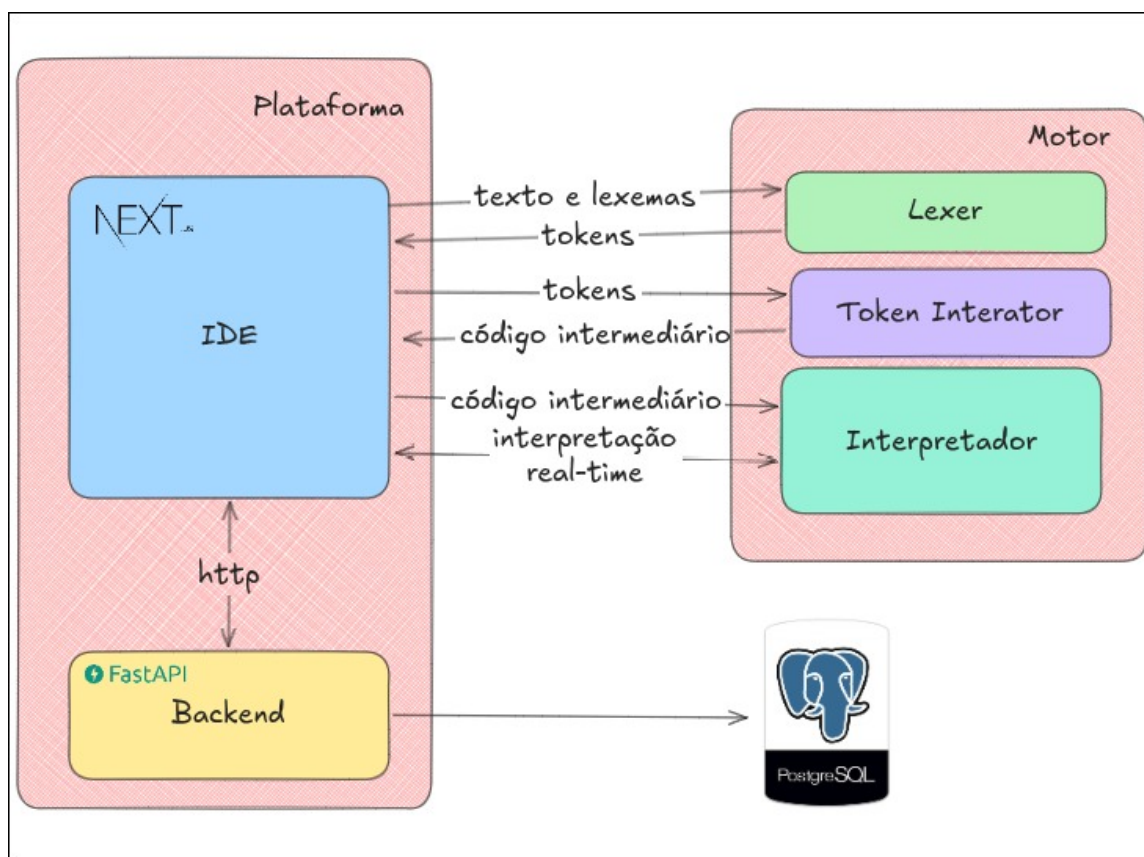
para análise, por meio de documentação, publicação acadêmica ou ferramenta disponível publicamente; e (iii) guardasse relação direta com ao menos uma das dimensões da solução proposta (a personalização da sintaxe, a execução interativa ou a visualização do processo de tradução). Cada ferramenta foi então examinada quanto à sua proposta, às suas características e às suas limitações, culminando na análise crítica que evidencia o diferencial da plataforma desenvolvida.

4.2 Modelagem da Arquitetura do Projeto

A organização do sistema foi concebida sob a ótica da Separação de Responsabilidades defendida por Martin (2019), dividindo o projeto em dois subprojetos independentes e versionados em um único repositório (*monorepo*): o pacote `compiler`, responsável pelo núcleo do interpretador, e o pacote `ide`, responsável pelo Ambiente de Desenvolvimento Integrado web e por toda a interação com o usuário. O isolamento entre esses módulos garante que o núcleo de tradução possa evoluir, ser testado e ser executado de maneira autônoma, sem qualquer acoplamento à camada de apresentação, atendendo à premissa de que regras centrais do domínio não devem depender de detalhes periféricos de entrega.

O fluxo de processamento entre os módulos foi estruturado em estágios sequenciais, conforme ilustra a Figura 4.1. O código-fonte digitado pelo usuário na IDE é transmitido, juntamente com a configuração corrente da linguagem (palavras-chave personalizadas, delimitadores de bloco, operadores em forma de palavra, literais booleanos e regras de finalização de instrução), ao analisador léxico do pacote `compiler`. A saída desse analisador, uma lista de *tokens*, alimenta o analisador sintático, que opera por descida recursiva e, de maneira concomitante, aciona o gerador de código intermediário. O conjunto de instruções de três endereços resultante é então recebido pelo interpretador, que executa o programa em tempo real diretamente no servidor da aplicação, repassando as operações de entrada e saída para o terminal embutido na interface.

Figura 4.1 – Fluxo de execução entre os módulos do interpretador dinâmico.



Fonte: elaborado pelo autor.

A comunicação entre a IDE e o núcleo do interpretador foi viabilizada por meio de rotas HTTP expostas pela aplicação Next.js. As rotas `/api/lexer` e `/api/intermediator` oferecem ao usuário visões intermediárias do processo de tradução, retornando, respectivamente, a tabela de *tokens* reconhecidos e a lista de instruções intermediárias geradas. A rota `/api/submissions/validate`, por sua vez, executa o ciclo completo de compilação e interpretação para fins de avaliação automática de exercícios, instanciando o analisador léxico, o iterador de *tokens* e o interpretador diretamente no processo do servidor. Essa rota também é responsável por interagir com o serviço de persistência, implementado em FastAPI sobre SQLAlchemy assíncrono, conforme caracterizado por FASTAPI (2026), mantendo a separação rígida entre a camada de regras de negócio e a infraestrutura de armazenamento.

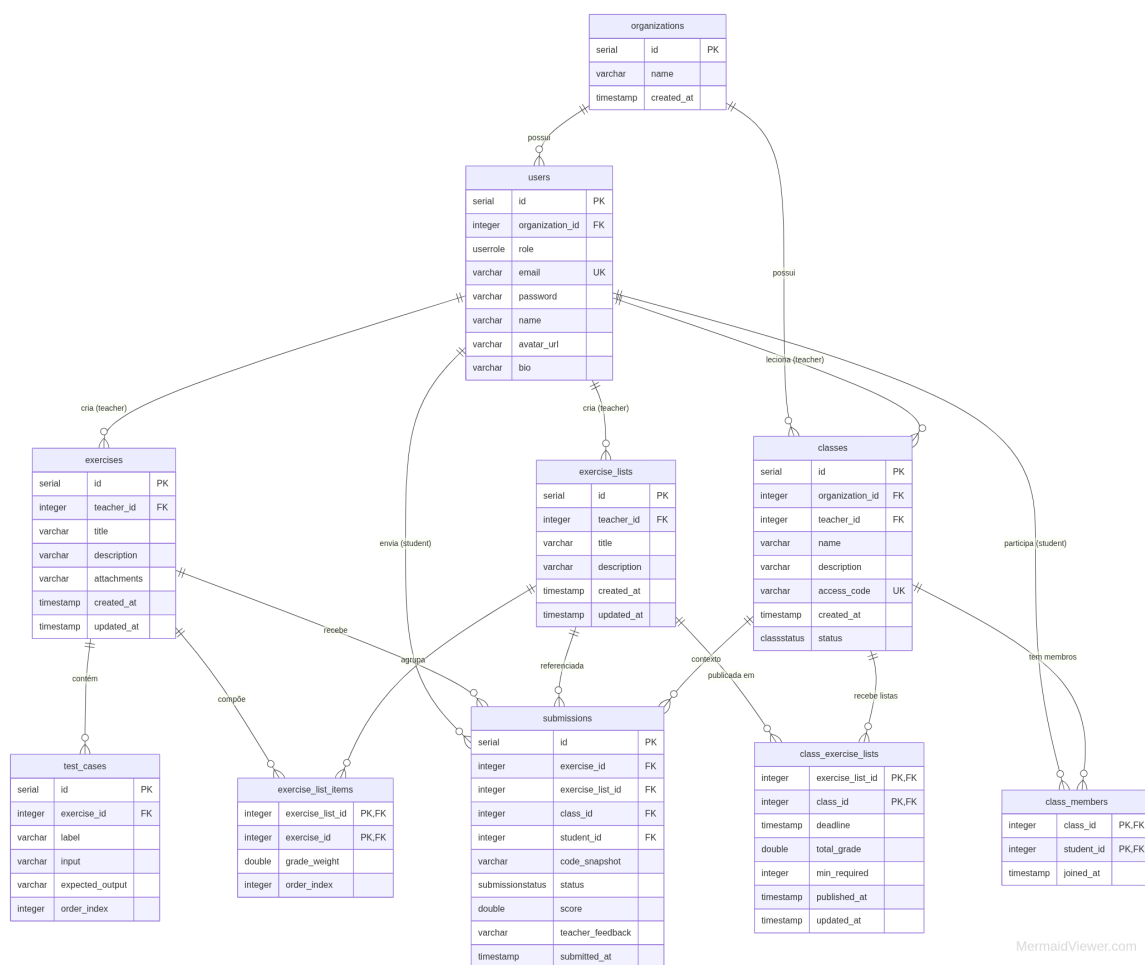
Para a construção dos componentes de análise, foi adotado o padrão de projeto *Factory* discutido por Silveira et al. (2012), materializado pela classe `LexerScannerFactory`. Esta abstração centraliza a decisão sobre qual analisador especializado deve ser invocado a cada caractere lido, encapsulando a complexidade de instanciação e reduzindo o acoplamento global

entre os *scanners* de comentários, identificadores, números, *strings* e símbolos. O mesmo princípio guiou a organização do analisador sintático, no qual cada não-terminal da gramática foi materializado em um arquivo independente, refletindo diretamente a estrutura formal da linguagem e favorecendo tanto a legibilidade quanto a testabilidade do código.

4.2.1 Modelagem do Banco de Dados

A camada de persistência da plataforma foi projetada em conformidade com os princípios de normalização relacional discutidos por Elmasri e Navathe (2015), adotando o gerenciador de banco de dados PostgreSQL e o esquema lógico ilustrado na Figura 4.2. As entidades foram separadas para reduzir redundâncias, preservar dependências funcionais e manter a integridade referencial entre instituições, usuários, turmas, exercícios, listas e submissões. O modelo organiza as entidades em torno de três eixos principais: a gestão institucional, representada pelas entidades *organizations*, *users* e *classes*; a gestão de conteúdo pedagógico, compreendendo *exercises*, *exercise_lists* e *test_cases*; e o registro de atividades dos estudantes, materializado pelas entidades *submissions* e pelas tabelas de associação *class_members*, *exercise_list_items* e *class_exercise_lists*.

Figura 4.2 – Esquema lógico do banco de dados da plataforma.



Fonte: elaborado pelo autor.

A entidade *organizations* representa as instituições de ensino cadastradas na plataforma, agrupando professores e alunos sob um contexto administrativo comum. A entidade *users* unifica os dois perfis de usuário da plataforma, professor e estudante, por meio de um atributo de papel (*role*), cujos valores possíveis são definidos pelo tipo enumerado *userrole*, mantendo uma única tabela de autenticação e evitando a duplicação de atributos compartilhados como nome, endereço de correio eletrônico e imagem de perfil. Cada usuário é obrigatoriamente associado a uma organização, garantindo o isolamento multilocatário dos dados.

A entidade *classes* representa as turmas criadas por professores e é identificada externamente por um código de acesso único (*access_code*), utilizado pelos estudantes para se matricular. O ciclo de vida de uma turma é controlado pelo tipo enumerado *classstatus*, que distingue turmas ativas, encerradas ou arquivadas. A relação de matrícula é mantida pela tabela associativa *class_members*, que registra o estudante e o instante de ingresso na turma.

Os exercícios individuais são cadastrados na entidade `exercises`, vinculados ao professor responsável pela sua criação. Cada exercício pode possuir múltiplos casos de teste, armazenados em `test_cases`, que registram a entrada fornecida ao programa e a saída esperada para fins de avaliação automática, conforme o ciclo de validação descrito na Seção 4.5. A relação de dependência entre exercício e casos de teste é implementada com exclusão em cascata, de modo que a remoção de um exercício elimine automaticamente seus respectivos casos de teste, preservando a integridade referencial do esquema.

As listas de exercícios são modeladas pela entidade `exercise_lists`, criadas pelo professor e compostas por itens representados pela tabela `exercise_list_items`. Esta tabela associativa, além de referenciar o exercício e a lista, armazena o peso da nota (`grade_weight`) e o índice de ordenação (`order_index`), permitindo que o professor controle tanto a distribuição de pontos quanto a sequência de apresentação dos itens aos estudantes. A publicação de uma lista para uma turma específica é gerenciada pela tabela `class_exercise_lists`, que registra o prazo de entrega (`deadline`), a nota total (`total_grade`), o número mínimo de exercícios exigidos (`min_required`) e os instantes de publicação e de última atualização, conferindo ao professor controle granular sobre a disponibilização do conteúdo.

A entidade `submissions` registra cada tentativa de envio realizada por um estudante, capturando um instantâneo imutável do código submetido (`code_snapshot`), o resultado da avaliação automática expresso pelo tipo enumerado `submissionstatus` e a pontuação calculada com base nos casos de teste aprovados. O campo de retroalimentação do professor (`teacher_feedback`) permite a complementação qualitativa da avaliação automática. A referência simultânea ao exercício, à lista de exercícios e à turma possibilita consultas agregadas de desempenho tanto por estudante quanto por turma ou por lista, sem exigir junções adicionais. Índices secundários foram definidos sobre os campos (`exercise_list_id`, `class_id`) e `student_id` para otimizar as consultas de maior frequência, como a listagem de submissões por turma e a visualização do histórico de um estudante, em conformidade com as diretrizes de desempenho discutidas por Elmasri e Navathe (2015).

O versionamento do esquema lógico é gerenciado pelo Alembic, cujo estado corrente de migração é rastreado pela tabela `alembic_version`. Essa abordagem permite a evolução incremental do banco de dados sem perda de dados em implantações sucessivas, em consonância com a estratégia de entrega contínua adotada no projeto.

4.2.2 Especificação das Rotas da API

A interface de comunicação entre os clientes e os serviços da plataforma envolve dois modelos distintos de invocação do compilador, combinados com um conjunto de rotas HTTP expostas pelo serviço de persistência. Uma decisão central de arquitetura foi a de que as fases de análise léxica e de geração de código intermediário, cujo resultado é exibido interativamente na IDE, são executadas inteiramente no navegador do usuário, sem qualquer requisição HTTP. Os *hooks* `useLexerAnalyse` e `useIntermediatorCode` importam e instanciam diretamente as classes `Lexer` e `TokenIterator` do pacote `compiler`, o que é viável porque esse pacote é escrito em TypeScript puro, sem dependências exclusivas de Node.js para essas operações. Essa abordagem elimina a latência de rede nas interações pedagógicas mais frequentes e mantém o servidor da IDE desonerado das requisições de análise.

A única rota HTTP do pacote `ide` diretamente relacionada ao compilador é `POST /api/submissions/validate`. Essa rota, executada no processo do servidor Next.js, recebe o código-fonte e a configuração corrente da linguagem, executa o ciclo completo de análise léxica, geração de código intermediário e interpretação, e avalia o resultado contra os casos de teste do exercício recuperados do serviço de persistência. O isolamento dessa operação em uma rota de servidor, e não no navegador, é motivado pela necessidade de controlar o tempo máximo de execução por meio de um mecanismo de *timeout* e de registrar a submissão de forma autenticada no banco de dados. Ao término da execução, a rota envia os resultados consolidados ao serviço FastAPI e devolve ao *front-end* a estrutura `TValidationResult`, contendo o status de cada caso de teste, a pontuação obtida e eventuais avisos de compilação.

As demais rotas HTTP da plataforma pertencem ao serviço de persistência, implementado em Python com o *framework* FastAPI e o ORM SQLAlchemy assíncrono. A documentação completa e interativa dessas rotas é gerada automaticamente pelo próprio *framework*, conforme descrito por FASTAPI (2026), e está disponível nos endereços <<https://api.dynamicinterpreter.com.br/docs>> (interface Swagger UI) e <<https://api.dynamicinterpreter.com.br/redoc>> (interface ReDoc). Essas rotas organizam-se nos seguintes grupos de recursos:

- **Autenticação** (`/auth`): gerencia o registro e o *login* de usuários, emitindo tokens JWT utilizados nas demais requisições protegidas;
- **Organizações** (`/organizations`): gerencia as instituições de ensino cadastradas na plataforma, às quais professores e alunos são vinculados;

- **Usuários** (`/users`): expõe operações de consulta e atualização de perfil para os dois papéis suportados, professor e estudante, com controle de acesso baseado no atributo `role`;
- **Turmas** (`/classes`): permite ao professor criar, editar, encerrar e arquivar turmas; a matrícula de estudantes é realizada por meio do código de acesso único da turma, gerenciado pela sub-rota `/classes/join`;
- **Exercícios** (`/exercises`): centraliza a criação e a manutenção dos enunciados de exercícios, incluindo seus casos de teste, com restrição de escrita ao professor responsável;
- **Listas de exercícios** (`/exercise-lists`): gerencia a composição e a ordenação dos exercícios em uma lista, com controle de pesos de nota por item;
- **Publicação de listas** (`/class-exercise-lists`): controla a associação de uma lista a uma turma, registrando prazo, nota total e mínimo de exercícios exigidos;
- **Submissões** (`/submissions`): armazena e recupera os registros de submissão dos estudantes, incluindo o instantâneo do código, a pontuação e a retroalimentação do professor.

Essa organização reflete diretamente a divisão arquitetural do projeto: o compilador opera localmente no navegador para as interações pedagógicas de baixa latência, em uma rota de servidor para a avaliação controlada de submissões, e o serviço FastAPI permanece inteiramente agnóstico em relação ao compilador, concentrando-se exclusivamente na gestão das entidades de domínio pedagógico. A relação completa dos grupos de recursos da API está reproduzida no Apêndice B.

4.3 Especificação da Linguagem e de seus Elementos Configuráveis

A linguagem suportada pelo interpretador foi projetada como um subconjunto didático de linguagens imperativas tipadas, suficientemente expressivo para abranger os conteúdos típicos das disciplinas introdutórias de programação, e simultaneamente flexível para que professores e alunos possam customizar suas regras superficiais. O conjunto base de funcionalidades contempla declarações de variáveis com tipagem estática, atribuições, expressões aritméticas, lógicas e relacionais, estruturas condicionais (`if/else` e `switch`), estruturas de repetição

(`for` e `while`), arranjos uni e multidimensionais com modos de alocação fixos e dinâmicos, e operações de entrada e saída padrão (`print` e `scan`).

A definição de quais aspectos da linguagem seriam exibidos como configuráveis foi guiada pelos achados empíricos de Stefik e Siebert (2013), segundo os quais a sintaxe tradicional derivada da família C atua como uma barreira significativa para estudantes iniciantes. Optou-se, portanto, por permitir a personalização precisamente daqueles elementos que, comprovadamente, oneram a carga cognitiva inicial: o vocabulário das palavras reservadas, os operadores lógicos e relacionais (que podem ser substituídos por palavras-chave em linguagem natural), os literais booleanos, os delimitadores de bloco (chaves ou indentação) e o terminador de instrução (ponto e vírgula opcional ou outro caractere arbitrário). Essas opções de configuração foram modeladas pela estrutura `LexerConfig`, transmitida ao analisador léxico a cada nova compilação e validada antes de qualquer processamento.

A validação prévia da configuração é tratada por funções utilitárias dedicadas, como `validateOperatorWordMap`, `validateBooleanLiteralMap`, `validateBlockDelimiters` e `validateStatementTerminatorLexeme`. Cada uma dessas funções verifica restrições léxicas elementares antes do início da análise, impedindo, por exemplo, que palavras-chave personalizadas colidam com identificadores válidos ou que delimitadores de bloco contenham caracteres inválidos. Essa estratégia preventiva atende aos princípios de detecção precoce de erros discutidos por Cooper e Torczon (2012), transformando inconsistências de configuração em mensagens claras antes mesmo da leitura do código-fonte. A Figura 4.3 apresenta a implementação das quatro funções de validação.

Figura 4.3 – Funções de validação prévia da configuração léxica da linguagem.

```

1 export function validateOperatorWordMap(
2   operatorWordMap: OperatorWordMap,
3   reserved: KeywordMap,
4   customKeywords: KeywordMap = {},
5   blockDelimiters?: LexerBlockDelimiters,
6 ): void {
7   const seenAliases = new Set<string>();
8
9   for (const alias of Object.values(operatorWordMap)) {
10    if (typeof alias !== "string") {
11      continue;
12    }
13
14    const normalizedAlias = alias.trim();
15    if (normalizedAlias.length === 0) {
16      continue;
17    }
18
19    if (!WORD_REGEX.test(normalizedAlias)) {
20      throw new Error("operator aliases must be identifier-like words");
21    }
22
23    if (seenAliases.has(normalizedAlias)) {
24      throw new Error("operator aliases cannot be duplicated");
25    }
26    seenAliases.add(normalizedAlias);
27
28    if (reserved[normalizedAlias] !== undefined) {
29      throw new Error("operator aliases cannot reuse reserved keywords");
30    }
31
32    if (customKeywords[normalizedAlias] !== undefined) {
33      throw new Error(
34        "operator aliases cannot conflict with keyword overrides",
35      );
36    }
37
38    if (
39      blockDelimiters &&
40      (normalizedAlias === blockDelimiters.open ||
41       normalizedAlias === blockDelimiters.close)
41    ) {
42      throw new Error("operator aliases cannot conflict with block delimiters");
43    }
44  }
45 }

```

(a) validateOperatorWordMap

```

1 export function validateBooleanLiteralMap(
2   booleanLiteralMap: BooleanLiteralMap,
3   reserved: KeywordMap,
4   customKeywords: KeywordMap = {},
5   operatorWordMap: OperatorWordMap = {},
6   blockDelimiters?: LexerBlockDelimiters,
7 ): void {
8   const seenAliases = new Set<string>();
9   const operatorAliases = new Set(
10     Object.values(operatorWordMap)
11       .filter((alias): alias is string => typeof alias === "string")
12       .map(alias => alias.trim())
13       .filter(alias => alias.length > 0),
14   );
15
16   for (const [slot, alias] of Object.entries(booleanLiteralMap)) {
17     if (typeof alias !== "string") {
18       continue;
19     }
20
21     const normalizedAlias = alias.trim();
22     if (normalizedAlias.length === 0) {
23       continue;
24     }
25
26     if (!WORD_REGEX.test(normalizedAlias)) {
27       throw new Error("boolean literal aliases must be identifier-like words");
28     }
29
30     if (seenAliases.has(normalizedAlias)) {
31       throw new Error("boolean literal aliases cannot be duplicated");
32     }
33     seenAliases.add(normalizedAlias);
34
35     if (reserved[normalizedAlias] !== undefined && normalizedAlias !== slot) {
36       throw new Error("boolean literal aliases cannot reuse reserved keywords");
37     }
38
39     if (operatorAliases.has(normalizedAlias)) {
40       throw new Error(
41         "boolean literal aliases cannot conflict with operator aliases",
42       );
43     }
44
45     if (
46       blockDelimiters &&
47       (normalizedAlias === blockDelimiters.open ||
48        normalizedAlias === blockDelimiters.close)
49     ) {
50       throw new Error(
51         "boolean literal aliases cannot conflict with block delimiters",
52       );
53     }
54   }
55 }

```

(b) validateBooleanLiteralMap

```

1 export function validateBlockDelimiters(
2   delimiters: LexerBlockDelimiters,
3   reserved: KeywordMap,
4 ): void {
5   const { open, close } = delimiters;
6
7   if (!WORD_REGEX.test(open) || !WORD_REGEX.test(close)) {
8     throw new Error("block delimiters must be identifier-like words");
9   }
10
11   if (open === close) {
12     throw new Error("block delimiters must be different");
13   }
14
15   if (reserved[open] !== undefined || reserved[close] !== undefined) {
16     throw new Error("block delimiters cannot reuse reserved keywords");
17   }
18 }

```

(c) validateBlockDelimiters

```

1 export function validateStatementTerminatorLexeme(
2   lexeme: string,
3   reserved: KeywordMap = {},
4   customKeywords: KeywordMap = {},
5   operatorWordMap: OperatorWordMap = {},
6   booleanLiteralMap: BooleanLiteralMap = {},
7   blockDelimiters?: LexerBlockDelimiters,
8 ): void {
9   if (lexeme.trim().length === 0) {
10     throw new Error("statement terminator cannot be empty");
11   }
12
13   if (/^\s/.test(lexeme)) {
14     throw new Error("statement terminator cannot contain whitespace");
15   }
16
17   if (lexeme === ";") {
18     throw new Error("statement terminator cannot reuse semicolon");
19   }
20
21   if ([...lexeme].some((char) => FIXED_TOKEN_CHARS.has(char))) {
22     throw new Error(
23       "statement terminator cannot reuse fixed operator or symbol characters",
24     );
25   }
26 }

```

(d) validateStatementTerminatorLexeme

Fonte: elaborado pelo autor.

4.4 Interpretador

O núcleo de tradução foi implementado em TypeScript, executado tanto via Node.js no servidor da IDE quanto pelo motor JavaScript do navegador em modos auxiliares de visualização. A escolha pela linguagem TypeScript permitiu o uso intensivo do sistema de tipos durante o

desenvolvimento, aproximando o processo de implementação dos benefícios de verificação estática descritos por Aho, Sethi e Ullman (1995) e Martin (2009), e contribuindo para a robustez das estruturas de dados que atravessam todas as fases do compilador.

4.4.1 Análise Léxica

A análise léxica foi implementada na classe `Lexer`, que percorre o código-fonte caractere por caractere e produz uma lista ordenada de instâncias da classe `Token`. Cada *token* carrega quatro atributos essenciais à fase seguinte e à emissão de erros: um identificador numérico (`type`), associado a uma das categorias léxicas suportadas pela linguagem, o lexema literalmente lido do código-fonte, a linha e a coluna em que o lexema iniciou-se. Essa estrutura está alinhada à distinção teórica entre lexema e *token* discutida por Nystrom (2021) e foi mantida estável ao longo de todo o desenvolvimento, pois constitui a interface entre o *front-end* e o restante do compilador. A decomposição de uma instrução em sua sequência de *tokens* segue o conceito ilustrado na Figura 2.2, apresentada no referencial teórico.

Os *tokens* foram organizados em sete grupos temáticos, a saber: aritméticos, lógicos, relacionais, atribuições, palavras reservadas, símbolos e literais, cada um deles definido em um módulo próprio sob o diretório `token/constants`. Essa segregação favorece a manutenção da tabela de *tokens* e habilita o uso de estilos visuais distintos por categoria na IDE, alinhados aos campos `TOKENS_STYLE` associados a cada grupo. A tabela completa de tipos é exportada também na forma invertida (`BY_ID`), o que viabiliza a tradução reversa de identificadores numéricos em descrições legíveis, fundamental para fins de depuração e visualização pedagógica.

A escolha por uma análise léxica determinística orientada por caracteres em vez de geradores automáticos como *Lex* foi motivada pela natureza dinâmica das palavras-chave da linguagem proposta. Como o vocabulário não é fixo em tempo de compilação, autômatos finitos pré-gerados perderiam a flexibilidade exigida pelo projeto. Em substituição, adotou-se o padrão *Factory* no diretório `lexer/scanners`, no qual a função `LexerScannerFactory.getInstance()` despacha o caractere corrente para o *scanner* apropriado: `comment.ts` para comentários de linha e de bloco, `identifier.ts` para identificadores e palavras-chave, `number.ts` para literais inteiros e em ponto flutuante, `string.ts` para literais de cadeia com sequências de escape, e `symbol-and-operator.ts` para os demais símbolos. Um *scanner* adicional, `statement-terminator.ts`, é instanciado dinamicamente conforme o terminador de instrução configurado pelo usuário, podendo assumir caracteres distintos do

ponto e vírgula tradicional ou mesmo ser desativado. A Tabela 4.1 sintetiza os analisadores especializados e suas respectivas responsabilidades.

Tabela 4.1 – Analisadores especializados (*scanners*) despachados pela `LexerScannerFactory`.

Scanner	Responsabilidade
<code>comment.ts</code>	Comentários de linha (<code>//</code>) e de bloco (<code>/* */</code>).
<code>identifier.ts</code>	Identificadores e palavras-chave, classificados conforme o vocabulário ativo na configuração corrente.
<code>number.ts</code>	Literais inteiros e em ponto flutuante.
<code>string.ts</code>	Literais de cadeia, incluindo o tratamento de sequências de escape.
<code>symbol-and-operator.ts</code>	Operadores e demais símbolos da linguagem.
<code>statement-terminator.ts</code>	Terminador de instrução configurável, instanciado dinamicamente conforme a configuração do usuário.

Fonte: elaborado pelo autor.

A função `buildEffectiveKeywordMap`, alimentada pela configuração de personalização proveniente da IDE, constrói em tempo de execução o mapeamento entre os lexemas das palavras-chave correntes e os identificadores numéricos canônicos dos *tokens*. Dessa forma, ao reconhecer um identificador no código-fonte, o analisador léxico verifica primeiro se o lexema corresponde a alguma palavra-chave ativa naquela configuração, classificando-o como `reserved` quando aplicável e como identificador comum caso contrário. Esse mecanismo concentra toda a flexibilidade linguística no analisador léxico, mantendo os estágios subsequentes inalterados ainda que o vocabulário superficial mude entre execuções.

A análise léxica também é responsável pelo modo de delimitação de blocos. Quando a IDE indica que o usuário optou por blocos baseados em indentação, em substituição às chaves, o *lexer* emite *tokens* sintéticos de `indent`, `dedent` e `newline` em pontos apropriados, aproveitando o registro contínuo de coluna e o tamanho de tabulação configurado pelo usuário. Esse comportamento foi inspirado nas linguagens estruturadas por indentação e permite que a etapa de análise sintática mantenha uma única gramática base, alternando entre blocos delimitados por chaves ou por indentação a partir do conjunto de *tokens* efetivamente gerado.

4.4.2 Análise Sintática

A análise sintática foi implementada por meio de um analisador descendente recursivo, materializado no diretório `grammar/syntax`. A escolha por esta estratégia, em detrimento das famílias *bottom-up* discutidas por Aho, Sethi e Ullman (1995), decorreu

de três fatores: a clareza pedagógica da correspondência direta entre regras de produção e funções TypeScript, a facilidade de emissão de mensagens de erro contextualizadas, e a aderência natural ao requisito de visualização passo a passo do processo de compilação dentro da IDE. Cada não-terminal da gramática foi materializado em um arquivo dedicado, como `stmt.ts`, `ifStmt.ts`, `forStmt.ts`, `whileStmt.ts`, `switchStmt.ts`, `declarationStmt.ts`, `ioStmt.ts` e `functionCallExpr.ts`, contendo, em comentários, a derivação formal correspondente.

A navegação sobre a sequência de *tokens* produzida pelo analisador léxico foi encapsulada na classe `TokenIterator`, que oferece operações de `peek`, `next`, `consume` e `match`, além de mecanismos de consulta ao modo de bloco e ao modo de tipagem corrente. O isolamento dessas operações em um único componente padroniza a forma como cada produção avança sobre o fluxo de *tokens* e protege o restante do analisador de possíveis erros de manipulação direta, como leituras fora dos limites ou consumos indevidos. Esta organização materializa o princípio do Padrão Iterador discutido por Silveira et al. (2012).

A gramática base foi cuidadosamente fatorada à esquerda e mantida livre de recursão à esquerda, conforme as restrições necessárias à análise preditiva discutidas por Aho, Sethi e Ullman (1995). Para acomodar as variações configuráveis pelo usuário, a função `consumeStmt Terminator` centraliza a verificação do terminador de instrução, podendo aceitar ponto e vírgula, lexemas alternativos definidos pelo usuário ou mesmo nenhum terminador, conforme a configuração ativa. O mesmo princípio governa o tratamento dos delimitadores de bloco: quando a IDE comunica ao analisador o modo indentado, o reconhecimento de `left_brace` e `right_brace` é substituído pelo casamento dos *tokens* sintéticos `indent` e `dedent` gerados pelo *lexer*, mantendo o restante da gramática inalterado.

4.4.3 Análise Semântica

A análise semântica foi entrelaçada com a fase sintática, na forma de tradução dirigida pela sintaxe descrita por Cooper e Torczon (2012). À medida que o analisador descendente reconhece uma construção válida, ele realiza imediatamente as verificações contextuais correspondentes, antes de prosseguir para a próxima produção. Essa estratégia evita a construção explícita e completa de uma árvore sintática intermediária, reduzindo o uso de memória e simplificando a integração com o gerador de código intermediário descrito na Subseção 4.4.4.

Foram contemplados três grupos principais de verificações semânticas. O primeiro

refere-se à resolução de nomes e à validação de escopo: cada identificador citado em uma expressão é confrontado com a tabela de símbolos corrente, e referências a variáveis não declaradas são reportadas como erros semânticos. O segundo grupo trata da consistência de tipos em expressões, atribuições, índices de *arrays* e parâmetros de *scan*; nesse último caso, optou-se pelo registro de um “hint” explícito de tipo (`int` ou `float`) na instrução de entrada, transferindo a coerção final para o interpretador. O terceiro grupo refere-se à aridade e à estrutura: chamadas de funções com número incompatível de argumentos, acessos a *arrays* com número de dimensões diferente do declarado e usos de `break` ou `continue` fora de contextos de repetição são todos sinalizados como erros semânticos contextualizados.

4.4.4 Geração de Código Intermediário

A representação intermediária adotada é uma variante de código de três endereços, conforme caracterizado por Aho, Sethi e Ullman (1995) e Cooper e Torczon (2012), materializada pela interface `Instruction`, contendo um operador `op`, um destino `result` e dois operandos `operand1` e `operand2`. O conjunto de operadores suportados é definido como uma união de tipos em `interpreter/constants.ts` e inclui as seis operações aritméticas (+, -, *, /, % e //), as três operações lógicas (||, && e !), as seis operações relacionais, atribuições, declarações simples e de *arrays*, instruções de acesso e atualização de *arrays* (`ARRAY_GET` e `ARRAY_SET`), saltos condicionais e incondicionais (`IF`, `JUMP` e `RETURN`), rótulos (`LABEL`) e chamadas de sistema (`CALL`).

A geração dessas instruções é realizada pela classe `Emitter`, instanciada por cada análise sintática. O *emitter* mantém dois contadores monotônicos responsáveis pela criação de variáveis temporárias (`__temp0`, `__temp1`, ...) e de rótulos (`__label0`, `__label1`, ...), ambos garantidos como únicos no escopo do programa traduzido. As expressões são decompostas em operações binárias elementares, conforme o exemplo clássico apresentado no referencial teórico, e o controle de fluxo das estruturas `if`, `while`, `for` e `switch` é traduzido em sequências de rótulos e saltos condicionais, eliminando a hierarquia das construções de alto nível em favor da representação linear esperada pelo interpretador.

A opção por gerar três endereços, em detrimento de representações mais abstratas como árvores sintáticas anotadas, foi motivada pelo propósito didático da plataforma. A linearidade das instruções emitidas permite que a IDE apresente, na interface, exatamente o fluxo de operações que será executado, tornando visível para o estudante o vínculo entre construções de alto nível

e suas decomposições elementares. Essa exposição direta da representação intermediária é uma característica diferencial da plataforma em relação às ferramentas analisadas nos trabalhos relacionados.

4.4.5 Interpretação e Tratamento de Erros em Tempo de Execução

A execução das instruções de três endereços foi implementada na classe `Interpreter`, projetada para operar sem pilha explícita de execução em sua versão base, com substituição direta de identificadores por seus valores correntes em uma tabela de símbolos. O ponteiro de instruções avança linearmente sobre a lista de `Instruction`, com exceção dos saltos, que reposicionam o ponteiro para o índice do rótulo associado. Para suportar funções, foi adicionada uma estrutura auxiliar de quadros de chamada (`CallFrame`), que armazena o endereço de retorno, o nome da variável destino e o escopo local da função invocada, permitindo a execução de chamadas e retornos sem comprometer a simplicidade da máquina virtual.

As operações de entrada e saída foram abstraídas por meio de funções de retorno (`stdout` e `stdin`) passadas ao construtor do interpretador. Essa decisão de projeto isola o motor de execução de qualquer detalhe de apresentação, permitindo que, na IDE, as saídas sejam direcionadas ao componente de terminal embutido, enquanto, nos testes automatizados, as mesmas operações sejam capturadas em *buffers* de *string*. Em conformidade com os princípios de baixo acoplamento e alta coesão discutidos por Martin (2019), essa abstração permitiu reutilizar o mesmo interpretador tanto em modo interativo quanto em modo de validação automatizada.

A leitura de valores via `scan` concentra um dos pontos mais delicados do tratamento de erros em tempo de execução. Como a entrada do usuário pode divergir do tipo esperado pela variável de destino (por exemplo, um valor textual digitado em uma leitura de inteiro), foi implementada uma rotina de coerção, `coerceValueForType`, complementada pela função `parseScanInput`, que inspeciona o “hint” semântico anotado na instrução e tenta converter a cadeia lida para o tipo declarado. Quando a conversão é impossível, em vez de propagar uma exceção genérica de *runtime*, o interpretador lança uma instância de `RuntimeError` contendo a posição original do comando no código-fonte e uma mensagem traduzida via o módulo `i18n`, permitindo que a IDE destaque visualmente a linha responsável e exiba o erro no idioma corrente do usuário.

Os erros estáticos detectados durante a compilação foram modelados pelo módulo `issue`, com três categorias distintas: `IssueError` representa falhas que interrompem

a tradução, `IssueWarning` representa alertas não fatais coletados durante a análise, e `IssueInfo` acumula mensagens informativas relevantes ao usuário. Toda ocorrência registra obrigatoriamente um código identificador, uma mensagem, a linha e a coluna do problema, permitindo que a IDE marque o trecho exato no editor e ofereça orientações pedagógicas associadas ao código do erro.

4.5 Integração entre a Plataforma e o Interpretador

A integração entre a plataforma web e o núcleo do interpretador foi materializada por meio do consumo direto do pacote `compiler` como dependência local do pacote `ide`, sem qualquer empacotamento intermediário em formato binário ou em servidor remoto separado. As importações realizadas na rota `/api/submissions/validate` acessam diretamente as classes `Lexer`, `TokenIterator` e `Interpreter`, bem como os tipos de configuração léxica, garantindo que qualquer alteração no compilador se propague à IDE sem etapas adicionais de implantação.

O ciclo de submissão de exercícios foi modelado em quatro passos sequenciais. Primeiro, a configuração corrente da linguagem é normalizada pela função `normalizeCompilerConfig`, que reconcilia o estado salvo do exercício com a configuração ativa na IDE e produz um descritor de gramática `IDEGrammarConfig` consistente. Em seguida, para cada caso de teste, instancia-se um analisador léxico configurado com esse descritor e executa-se a tradução completa do código submetido. Posteriormente, o interpretador é instanciado com funções `stdout` e `stdin` ligadas a *buffers* controlados, permitindo que a saída produzida seja comparada com a saída esperada e que a entrada simulada do caso de teste seja consumida sequencialmente. Por fim, o resultado consolidado de cada caso é encapsulado em uma estrutura `TTestCaseResult` e agregado em um `TValidationResult`, devolvido ao *front-end* para apresentação ao usuário.

A persistência de exercícios, turmas e submissões foi mantida em um serviço externo, implementado em Python sobre o *framework* FastAPI e o ORM SQLAlchemy assíncrono, com versionamento de *schema* via Alembic, em conformidade com os princípios de abstração de persistência discutidos por Martin (2019). Essa separação garante que o núcleo de compilação e interpretação permaneça agnóstico em relação à camada de armazenamento, podendo ser executado, inclusive, sem qualquer dependência de banco de dados nas execuções locais e nos testes automatizados.

4.6 Estratégia de Testes Automatizados

A validação contínua das fases do compilador foi sustentada por uma suíte de testes automatizados construída com o *framework* Vitest, organizada no diretório `packages/compiler/src/tests` e estruturada de modo a refletir a topologia interna do projeto. Os testes foram subdivididos em três camadas: testes de *tokens*, que verificam a estabilidade dos identificadores numéricos e dos agrupamentos por categoria; testes de *lexer*, que exercitam cada *scanner* especializado em isolamento, abrangendo comentários, literais numéricos, cadeias com escapes, terminadores configuráveis, delimitadores de bloco, operadores em forma de palavra e geração de *tokens* de indentação; e testes de gramática, que validam a aceitação ou rejeição de programas inteiros sob distintas configurações de tipagem, modos de bloco e variantes do comando `switch`.

A adoção do TDD discutida por Aniche (2012) mostrou-se particularmente útil em construções como o terminador de instrução configurável e o modo de blocos por indentação, em que pequenas alterações na lógica do *lexer* poderiam quebrar inadvertidamente programas que antes compilavam corretamente. O ciclo iterativo do TDD, composto pelas etapas de escrita do teste, implementação mínima e refatoração, conforme ilustra a Figura 2.3, foi seguido sistematicamente em cada nova funcionalidade do compilador. Em conformidade com a pirâmide de testes discutida por Martin (2019), a maior parte da suíte concentra-se no nível unitário, garantindo execuções rápidas e *feedback* imediato, enquanto os testes de gramática operam como testes de integração internos ao compilador, verificando o comportamento agregado das fases léxica, sintática e de geração de código intermediário.

Os mesmos testes automatizados são executados localmente durante o desenvolvimento e também acionados pelo *pipeline* de integração contínua antes da incorporação de qualquer mudança ao ramo principal do repositório, assegurando que as características essenciais do compilador, a saber, a estabilidade dos identificadores de *tokens*, a correção semântica das instruções intermediárias geradas e a equivalência de comportamento entre as variantes configuráveis da linguagem, sejam preservadas ao longo de toda a evolução incremental do projeto. A cada nova funcionalidade, a suíte é executada integralmente, e não apenas sobre o trecho recém-adicionado, de modo que regressões em fases anteriores sejam detectadas de imediato. Os casos de teste e o respectivo relatório de execução estão reunidos no Apêndice C.

REFERÊNCIAS

- ADAPTWORKS. **Agile: Desenvolvimento de Software com Entregas Frequentes e Foco no Valor de Negócio**. São Paulo: Casa do Código, 2012.
- AHO, A. V.; SETHI, R.; ULLMAN, J. D. **Compiladores: princípios, técnicas e ferramentas**. Rio de Janeiro: LTC, 1995. Tradução de Daniel de Ariosto Pinto.
- ALCANTARA, F. **Tabela de Símbolos**. 2024. Disponível em: <<https://frankalcantara.com/lf/11-tabelaSimbolos.html>>.
- ANICHE, M. **Test-Driven Development: Teste e Design no Mundo Real**. São Paulo: Casa do Código, 2012.
- BENEDETTI, T. Como escolher a melhor plataforma educativa em 2025. **Blog TutorMundi**, 2025. Disponível em: <<https://tutormundi.com/blog/>>. Acesso em: 2026.
- COOPER, K. D.; TORCZON, L. **Engineering a Compiler**. 2. ed. Waltham: Elsevier / Morgan Kaufmann, 2012.
- DASROOT. **Python Flask vs FastAPI vs Django: Framework Comparison 2026**. 2026. Disponível em: <<https://dasroot.net>>. Acesso em: 07 abr. 2026.
- DUNOSSAURO, E. **FastAPI do Zero!** 2026. Disponível em: <<https://dunossauro.github.io/fastapi-do-zero/>>. Acesso em: 07 abr. 2026.
- ELMASRI, R.; NAVATHE, S. B. **Fundamentals of Database Systems**. 7. ed. Hoboken: Pearson, 2015.
- FASTAPI. **Recursos e Documentação Oficial do FastAPI**. 2026. Disponível em: <<https://fastapi.tiangolo.com/>>. Acesso em: 07 abr. 2026.
- FERREIRA, B.; SILVA, C. L.; RIBEIRO, F. d. C. L. Um estudo de caso da ferramenta beecrowd como tecnologia da informação e comunicação na aprendizagem de algoritmos. **ForScience**, v. 13, n. 2, 2025. Disponível em: <<https://doi.org/10.29069/forscience.2025v13n2.e1313>>.
- FIRMINIQ. **Test-Driven Development (TDD) and its Impact on Quality Assurance (QA)**. 2024. Disponível em: <<https://firminiq.com/test-driven-development-tdd-and-its-impact-on-quality-assurance-qa/>>.
- GARCIA, L. **Beecrowd: como desenvolver habilidades praticando programação**. Alura, 2024. Disponível em: <<https://www.alura.com.br/artigos/beecrowd-como-desenvolver-habilidades-praticando-programacao>>. Acesso em: 07 abr. 2026.
- GOMES, D. **O que é Portugol Studio?** DevMedia, 2020. Disponível em: <<https://www.devmedia.com.br/o-que-e-portugol-studio/40764>>. Acesso em: 07 abr. 2026.
- LADNER, R. E. **Learn About the New Quorum Studio**. Computer Science Teachers Association (CSTA), 2019. Disponível em: <<https://csteachers.org/learn-about-the-new-quorum-studio/>>. Acesso em: 07 abr. 2026.

MÃO NO CÓDIGO. BEECROWD: Todo PROGRAMADOR deveria conhecer | BEECROWD (antigo Uri Online Judge). 2022. Vídeo do YouTube. Disponível em: <<https://www.youtube.com/watch?v=ExemploUrl>>. Acesso em: 07 abr. 2026.

MARTIN, R. C. **Código Limpo: Habilidades Práticas do Agile Software.** Rio de Janeiro: Alta Books, 2009.

MARTIN, R. C. **The Clean Coder: A Code of Conduct for Professional Programmers.** Upper Saddle River: Prentice Hall, 2011.

MARTIN, R. C. **Arquitetura Limpa: O Guia do Artesão para Estrutura e Design de Software.** Rio de Janeiro: Alta Books, 2019.

MARTINS, C. **Portugol Studio: Guia Completo de Como Usar a Plataforma.** HostGator, 2025. Disponível em: <<https://www.hostgator.com.br/blog/portugol-studio/>>. Acesso em: 07 abr. 2026.

Mind Makers. 10 tendências em edtech para 2026: o que a escola precisa saber. **Blog Mind Makers**, 2025. Disponível em: <<https://mindmakers.com.br/blog/>>. Acesso em: 2026.

NYSTROM, R. **Crafting Interpreters.** [S.l.]: Genever Benning, 2021. ISBN 978-0990582939.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: uma Abordagem Profissional.** 8. ed. Porto Alegre: AMGH, 2016.

PRICE, A. M. A.; TOSCANI, S. S. **Implementação de Linguagens de Programação: Compiladores.** Porto Alegre: Sagra Luzzatto, 2001. (Série Livros Didáticos).

QUORUM. **The Quorum Programming Language: Documentation and Curriculum.** 2025. Disponível em: <<https://quorumlanguage.com/>>. Acesso em: 07 abr. 2026.

SABBAGH, R. **Scrum: Gestão ágil para projetos de sucesso.** São Paulo: Casa do Código, 2014.

SILVA, D. d. S. et al. Minerando dados de um juiz on-line para prever a evasão de estudantes em disciplinas introdutórias de programação. In: **Anais do XXXI Simpósio Brasileiro de Informática na Educação (SBIE).** Porto Alegre: Sociedade Brasileira de Computação, 2020. p. 1343–1352.

SILVA, M. L. d. **Laila: Uma Plataforma Web para Auxílio ao Ensino de Compiladores.** Monografia (Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação)) — Instituto Federal de Educação, Ciência e Tecnologia do Ceará, Aracati, 2021.

SILVEIRA, P. et al. **Introdução à arquitetura e design de software.** São Paulo: Casa do Código, 2012.

SOMMERVILLE, I. **Engenharia de Software.** 10. ed. São Paulo: Pearson Education do Brasil, 2018.

SOUZA, D. M.; BATISTA, M. H. d. S.; BARBOSA, E. F. Problemas e dificuldades no ensino e na aprendizagem de programação: um mapeamento sistemático. **Revista Brasileira de Informática na Educação**, v. 24, n. 1, p. 39–52, 2016.

STEFIK, A.; SIEBERT, S. An empirical investigation into programming language syntax. **ACM Transactions on Computing Education**, ACM, v. 13, n. 4, 2013.

UEYAMA, R. **chibicc: A small C compiler**. 2020. Disponível em: <<https://github.com/rui314/chibicc>>. Acesso em: 07 abr. 2026.

UNIVALI. **Portugol Studio - LITE**. Laboratório de Inovação Tecnológica na Educação, 2026. Disponível em: <<https://univali-lite.github.io/Portugol-Studio/>>. Acesso em: 07 abr. 2026.

A PLANO DE TRABALHO: DIVISÃO DE RESPONSABILIDADES

Este apêndice detalha a divisão de responsabilidades entre os dois autores, conforme a pré-proposta aprovada e o plano de trabalho aprovado pelo Colegiado do Curso de Engenharia de Computação do Centro Federal de Educação Tecnológica de Minas Gerais, Unidade Leopoldina.

A.1 Atividades Desenvolvidas em Regime Colaborativo

As atividades listadas a seguir, de natureza estruturante ou situadas na fronteira entre os dois subsistemas, foram conduzidas de forma colaborativa por Victor de Souza Vilela da Silva e Igor Lamoia Queiroz:

- Revisão bibliográfica sobre interpretadores, compiladores e ferramentas educacionais correlatas;
- Definição dos requisitos funcionais e não funcionais e da arquitetura geral do sistema;
- Definição dos contratos de comunicação entre o núcleo do interpretador e a plataforma web e a integração entre os módulos;
- Revisão mútua de código e tomada de decisões arquiteturais nas etapas de refatoração e evolução do sistema.

A.2 Eixos de Aprofundamento Individual

A partir da base técnica construída em conjunto, cada autor aprofundou sua análise acadêmica em eixos específicos, conforme previsto na pré-proposta.

A.2.1 Victor de Souza Vilela da Silva (esta monografia)

- Detalhamento das fases de análise léxica e do sistema de varredura por escâneres especializados (*factory pattern*);
- Descrição aprofundada da análise sintática por descida recursiva e da correspondência entre as produções do analisador e a gramática formal da linguagem Java--;
- Implementação e documentação da geração de código de três endereços: instrução a instrução, tabela de temporários e tabela de rótulos;

- Especificação e implementação dos mecanismos de injeção dinâmica de regras gramaticais no interpretador, que permitem ao núcleo reconfigurar o vocabulário e as regras estruturais a cada execução;
- Análise do funcionamento do interpretador, da tabela de símbolos e do despacho das chamadas de sistema de entrada e saída;
- Estudos de caso centrados na validade técnica do núcleo de tradução e no comportamento do interpretador frente a programas de teste elaborados para cobrir as construções da linguagem.

A.2.2 Igor Lamoia Queiroz (documento complementar)

- Detalhamento da arquitetura da aplicação web (Next.js, Pages Router) e da integração com o Monaco Editor para edição de código com realce de sintaxe dinâmico;
- Documentação do assistente de personalização da linguagem (*wizard*), incluindo o modelo de etapas, a lógica de validação preventiva dos delimitadores de bloco e o mecanismo de pré-visualização em tempo real;
- Análise da experiência de uso da plataforma, dos contextos React que governam o estado da aplicação e das mediações pedagógicas oferecidas pela interface;
- Descrição da infraestrutura de submissão e correção automatizada de exercícios e do sistema de gerenciamento de turmas;
- Estudos de caso centrados na usabilidade, no engajamento dos estudantes e na retroalimentação coletada em ambiente educacional.

B DOCUMENTAÇÃO DA API

Este apêndice reúne a relação dos recursos expostos pela API da plataforma. A documentação completa e interativa é gerada automaticamente pelo *framework* FastAPI e está disponível *online* nas interfaces Swagger UI (<<https://api.dynamicinterpreter.com.br/docs>>) e ReDoc (<<https://api.dynamicinterpreter.com.br/redoc>>). A Tabela B.1 sintetiza os grupos de recursos e suas responsabilidades.

Tabela B.1 – Grupos de recursos da API da plataforma.

Recurso	Responsabilidade
POST /api/submissions/validate	Executa o ciclo completo de compilação e interpretação do código submetido e o avalia contra os casos de teste do exercício (rota do pacote <i>ide</i>).
/auth	Registro e <i>login</i> de usuários, com emissão de <i>tokens</i> JWT.
/organizations	Gestão das instituições de ensino cadastradas.
/users	Consulta e atualização de perfil, com controle de acesso por <i>role</i> .
/classes	Criação, edição, encerramento e arquivamento de turmas; matrícula via /classes/join.
/exercises	Criação e manutenção de enunciados e respectivos casos de teste.
/exercise-lists	Composição, ordenação e atribuição de pesos aos exercícios de uma lista.
/class-exercise-lists	Publicação de listas para turmas, com prazo, nota total e mínimo de exercícios.
/submissions	Armazenamento e recuperação das submissões dos estudantes, com pontuação e retroalimentação.

Fonte: elaborado pelo autor.

C CASOS DE TESTE AUTOMATIZADOS

Este apêndice resume a suíte de testes automatizados do núcleo do compilador, construída com o *framework* Vitest. A suíte reúne **215 casos** distribuídos em 22 arquivos, organizados de modo a refletir as camadas internas do projeto, conforme a Tabela C.1. O conjunto completo dos casos e o respectivo relatório de execução estão disponíveis no repositório do projeto (<<https://github.com/igorlamoia/ts-compiler-for-java>>, em `packages/compiler/src/tests`); à data desta qualificação, a totalidade dos casos é aprovada.

Tabela C.1 – Distribuição dos casos de teste por camada do compilador.

Camada	Arquivos	Casos	Escopo
Tokens	1	8	Estabilidade dos identificadores numéricos e dos agrupamentos por categoria.
Análise léxica	9	56	<i>Scanners</i> em isolamento: comentários, literais numéricos e de cadeia, terminador configurável, delimitadores de bloco, operadores em forma de palavra e geração de <i>tokens</i> de indentação.
Análise sintática e semântica	9	138	Aceitação ou rejeição de programas sob distintos modos de tipagem e de bloco, variantes do <code>switch</code> e verificação de tipos.
Interpretação	2	13	Execução do código de três endereços e sessões de depuração.
Total	22	215	

Fonte: elaborado pelo autor.